PulseOps por dentro: guia de código, decisões e preparação para entrevistas

Sumário

Identificação e como usar este guia

- Pré-requisitos
- Caminho de estudo recomendado
- Convenções

1 Parte A — Problema e produto

- 1.1 O que é uma API
- 1.2 O que é monitoramento
- 1.3 Lenta não é o mesmo que indisponível
- 1.4 O que é um incidente
- 1.5 O que cada funcionalidade resolve
- 1.6 Cenário fictício e limites do projeto

2 Parte B — Execução passo a passo

- 2.1 Terminal e diretórios
- 2.2 Instalação das ferramentas
- 2.3 Baixar o código
- 2.4 Ambientes e dependências
- 2.5 Variáveis de ambiente
- 2.6 Docker e Compose
- 2.7 Banco e migrações
- 2.8 Criar a operadora
- 2.9 Iniciar, usar e parar
- 2.10 Como conferir se está funcionando
- 2.11 O laboratório de falhas
- 2.12 Rodar os testes
- 2.13 Erros comuns e diagnóstico

3 Parte C — Arquitetura

- 3.1 Os processos
- 3.2 Responsabilidade de cada pasta e arquivo
- 3.3 Rota, schema, entidade e serviço
- 3.4 Caminho de uma requisição
- 3.5 Caminho de uma verificação
- 3.6 Caminho de um incidente
- 3.7 Caminho de uma notificação
- 3.8 Por que nenhuma fila externa

4 Parte E — Fundamentos de Python aplicados

- 4.1 Tipos
- 4.2 Coleções
- 4.3 Funções e argumentos
- 4.4 Imports e módulos
- 4.5 Classes e instâncias
- 4.6 Encapsulamento e composição

4.7 Protocolos (interfaces)
4.8 Exceções
4.9 Context managers
4.10 Tipagem
4.11 async/await
4.12 Concorrência e seus limites
4.13 Injeção de dependências
5 Parte F — Banco de dados e SQL
5.1 Modelo relacional
5.2 Chaves primárias e estrangeiras
5.3 Constraints: o banco como segunda linha de defesa
5.4 Índices
5.5 ORM: tabelas como classes Python
5.6 O SQL que realmente roda
5.7 Transações, commit e rollback
5.8 Migrações com Alembic
5.9 Concorrência
5.10 Resumo da parte F
6 Parte G — Testes, segurança e confiabilidade
6.1 Teste unitário e teste de integração
6.2 Fixtures e dublês
6.3 Por que PostgreSQL de verdade, e não SQLite
6.4 Testes de regressão
6.5 Cobertura
6.6 SSRF: um exemplo seguro
6.7 Autenticação, sessão e CSRF
6.8 Retentativas e duplicidade
6.9 Limitações conhecidas
7 Parte H — Perguntas de entrevista
7.1 H1. Por que FastAPI?
7.2 H2. Onde há orientação a objetos?
7.3 H3. Por que composição?
7.4 H4. O que acontece quando o worker reinicia?
7.5 H5. Como impedir incidentes duplicados?
7.6 H6. Por que uma sessão SQLAlchemy não deve ser compartilhada entre tarefas?
7.7 H7. Por que não executar tudo na rota?
7.8 H8. Como funciona a transação?
7.9 H9. Qual a diferença entre lentidão e falha?
7.10 H10. Como calcular métricas sem dados?
7.11 H11. O que ocorre se o webhook cair?
7.12 H12. Por que a entrega pode se repetir?
7.13 H13. Como o projeto trata SSRF?
7.14 H14. Qual a limitação de escala?

7.15 H15. O que você mudaria numa versão comercial?
7.16 H16. Por que API e worker em processos separados?
7.17 H17. Por que async e não threads?
7.18 H18. Como o worker sabe o que verificar e quando?
7.19 H19. O que é o run_key?
7.20 H20. Como impedir que um check comece antes do anterior terminar?
7.21 H21. Como o painel mostra que o worker parou?
7.22 H22. Como testar tempo sem `sleep`?
7.23 H23. Por que PostgreSQL de verdade nos testes?
7.24 H24. Como a senha é guardada?
7.25 H25. Como funciona a proteção contra CSRF?
7.26 H26. Como o p95 é calculado?
7.27 H27. Por que paginação por cursor?
7.28 H28. O que acontece se a configuração mudar durante um check?
7.29 H29. Qual foi o bug mais interessante?
7.30 H30. Como as migrações são validadas?
7.31 H31. O que é o padrão outbox?
7.32 H32. Como os logs evitam vazar segredos?
7.33 H33. Como o CI está configurado?
7.34 H34. Qual foi o papel da IA no projeto?
8 Parte I — Como apresentar o projeto
8.1 Explicação de 30 segundos
8.2 Explicação de 2 minutos
8.3 Roteiro de demonstração de 5 minutos
8.4 Descrição para currículo
8.5 O que mostrar primeiro a um recrutador
8.6 Afirmações que exigem evidência ou estudo antes
9 Parte J — Aprendizagem ativa
9.1 Nível 1 — Ler e localizar
9.2 Nível 2 — Prever o resultado antes de executar
9.3 Nível 3 — Alterar uma regra e atualizar os testes
9.4 Nível 4 — Implementar uma pequena melhoria
9.5 Nível 5 — Diagnosticar sem olhar a resposta
9.6 Checklist de compreensão
10 Gabarito comentado da parte J
10.1 Nível 1
10.2 Nível 2
10.3 Nível 3
10.4 Nível 4
10.5 Nível 5

Identificação e como usar este guia

Item	Valor
Material	*PulseOps por dentro: guia de código, decisões e preparação para entrevistas*
Versão do material	1.0
Data de geração	2026-10-22
Commit do código explicado	`a46ee8835aacde9f5b5bb40a5a516e23d98a0e76`
Volumes	Volume 1 (este): partes A, B, C, E, F, G, H, I e J, e o gabarito da parte J. Volume 2: parte D, código explicado linha a linha
Autora do projeto	Amanda da Mata

Sobre o commit. Os trechos de código deste guia foram copiados por um script (`study/build.py`) dos arquivos do repositório no commit `a46ee88`. O próprio PDF e a fonte dele (`study/`) foram commitados **depois**, em commits que só acrescentam documentação; por isso o hash acima é o do código explicado, não o do commit que contém o PDF. O script se recusa a gerar o material se algum arquivo de código tiver mudado desde esse commit.

Como o projeto foi feito. O PulseOps foi implementado com o Claude Code (assistente de IA) a partir de uma especificação escrita por Amanda. Este guia existe para que Amanda estude o código, entenda cada decisão e consiga explicá-la com as próprias palavras. Ele não afirma que ela já domina esses assuntos: a parte H indica, em cada pergunta de entrevista, o limite do que dá para afirmar.

Pré-requisitos

- Saber abrir um terminal e digitar comandos (a parte B explica cada um).
- Ter lido ao menos uma introdução de Python: variáveis, `if`, `for`, funções. A parte E retoma esses conceitos no código do PulseOps.
- Docker instalado, para executar o projeto. Para os testes, também o uv.
- Não é preciso saber FastAPI, SQL ou Docker antes: o guia explica o que cada coisa faz no momento em que aparece.

Caminho de estudo recomendado

1. **Primeira semana: o produto.** Leia a parte A, execute a parte B até ver o painel funcionando e rode `scripts/demo.sh`. Observe o painel enquanto o roteiro roda.
2. **Segunda semana: a regra central.** Leia a parte C e, no volume 2, o arquivo `domain/health.py` e o teste `tests/unit/test_health.py`. Faça os exercícios J1 a J11 da parte J.
3. **Terceira semana: um check do começo ao fim.** No volume 2, siga `worker/runner.py` → `services/processing.py` → `checks/checker.py` → `security/transport.py`. Leia a parte F (banco) junto.
4. **Quarta semana: alertas, API e painel.** `services/alerts.py`, `notifications/webhook.py`, `web/`. Leia a parte G.

5. **Quinta semana: entrevistas.** Parte H em voz alta, parte I, e os exercícios restantes da parte J sem olhar o gabarito.

Em cada etapa, mude algo pequeno, rode os testes e veja o que quebra. Ler sem executar rende pouco.

Convenções

- Nomes de código (variáveis, funções, arquivos, endpoints) aparecem `assim` e em inglês, como no repositório. O texto é em português.
- "Linha 42" sempre se refere à numeração do arquivo no commit acima.
- Os blocos "SQL enviado ao PostgreSQL" foram capturados durante a execução dos testes (`study/captura_sql.py`), não escritos à mão.
- **Usado no PulseOps** marca recursos que aparecem no código. **Só para comparação** marca conceitos citados para contraste, que o projeto não usa.

1 Parte A — Problema e produto

1.1 O que é uma API

Um programa conversa com outro por meio de uma **API** (Application Programming Interface, interface de programação de aplicações). Em uma API web, essa conversa usa o protocolo **HTTP**: quem pede (o *cliente*) envia uma **requisição** e quem atende (o *servidor*) devolve uma **resposta**.

Uma requisição HTTP tem:

- um **método**, que diz a intenção: `GET` (ler), `POST` (criar), `PUT` (substituir), `PATCH` (alterar parte), `DELETE` (apagar), `HEAD` (como `GET`, mas sem o corpo da resposta);
- uma **URL**, como `https://api.exemplo.com/pedidos?pagina=2`, com esquema (`https`), host (`api.exemplo.com`), porta (443, implícita no `https`), caminho (`/pedidos`) e *query string* (`?pagina=2`);
- **cabeçalhos** (*headers*), pares nome/valor como `Content-Type: application/json`;
- opcionalmente um **corpo**, por exemplo um JSON.

A resposta traz um **código de status** de três dígitos, cabeçalhos e, às vezes, um corpo. Os códigos se agrupam pela centena:

Faixa	Significado	Exemplos
2xx	deu certo	200 OK, 201 Created, 204 No Content
3xx	redirecionamento	301 Moved Permanently, 302 Found
4xx	erro de quem pediu	400 Bad Request, 401 Unauthorized, 404 Not Found, 422 Unprocessable Entity
5xx	erro do servidor	500 Internal Server Error, 503 Service Unavailable

O PulseOps aparece dos dois lados: ele **é** uma API (o painel e as rotas `/api/v1/...`) e ele **consome** APIs (faz requisições aos endereços monitorados e envia webhooks).

1.2 O que é monitoramento

Monitorar uma API é perguntar a ela, de tempos em tempos, "você está respondendo?", e guardar as respostas. No PulseOps, cada pergunta se chama **check** (verificação). Um **monitor** é a configuração de uma pergunta recorrente: qual URL, a cada quantos segundos, quanto tempo esperar, quais status contam como "deu certo" e quantas falhas seguidas confirmam um problema.

Esse tipo é chamado de **monitoramento ativo** (ou *sintético*): o monitor inicia o contato. No **monitoramento passivo**, o serviço é que avisa que está vivo; o Healthchecks, comparado na pesquisa de mercado, funciona assim. O PulseOps só faz monitoramento ativo de HTTP/HTTPS.

1.3 Lenta não é o mesmo que indisponível

Uma resposta pode chegar com status 200 e mesmo assim demorar 3 segundos. O serviço está **disponível** (respondeu e respondeu certo), mas está **lento** para quem usa. Tratar as duas coisas como iguais causa dois problemas:

- se lentidão contasse como falha, um pico de carga abriria incidentes e dispararia alertas de "fora do ar" para um serviço que está no ar;
- se lentidão fosse ignorada, a equipe só descobriria a degradação quando ela virasse queda.

Por isso o PulseOps classifica um check com status aceito e tempo acima do limite como **lento** (`slow`). O monitor fica **degradado** (amarelo), mas nenhum incidente é aberto (ADR 0004). Já timeout, erro de conexão, erro TLS ou status fora da lista aceita são **falhas** e contam para abrir incidente.

O **timeout** fica na fronteira: se a resposta não chega dentro do prazo configurado (por exemplo, 5 s), o PulseOps desiste e conta uma falha, porque do ponto de vista de quem usa o serviço "não respondeu".

1.4 O que é um incidente

Um **incidente** é um período em que o serviço é considerado indisponível, com começo, confirmação, fim e evidências. Um check isolado com erro não é um incidente: redes falham por instantes. O PulseOps só abre um incidente depois de `failure_threshold` falhas **seguidas** (3 por padrão) e só o encerra depois de `recovery_threshold` sucessos **seguidos** (2 por padrão).

Exemplo com os valores padrão (é o exemplo obrigatório da especificação, testado em `tests/unit/test_health.py`):

Check	Resultado	Falhas seguidas	Sucessos seguidos	Saúde	Incidente
1	falha	1	0	degradado	nenhum ("Falha 1 de 3")
2	falha	2	0	degradado	nenhum ("Falha 2 de 3")
3	sucesso	0	1	saudável	nenhum (contagem zerada)
4	falha	1	0	degradado	nenhum
5	falha	2	0	degradado	nenhum
6	falha	3	0	indisponível	**abre**; início = check 4, confirmação = check 6
7	falha	4	0	indisponível	o **mesmo** incidente é atualizado

Check	Resultado	Falhas seguidas	Sucessos seguidos	Saúde	Incidente
8	sucesso	0	1	indisponível	continua aberto (“Sucesso 1 de 2”)
9	sucesso	0	2	saudável	**resolvido**

Repare em duas datas diferentes: o incidente **começou** na primeira falha da sequência (check 4), mas só foi **confirmado** no check 6. As duas ficam gravadas.

1.5 O que cada funcionalidade resolve

Problema da equipe (cenário fictício)	Funcionalidade	Onde está
“Descobrimos quando alguém reclama”	Checks periódicos feitos pelo worker	`worker/runner.py`, `checks/checker.py`
“Não sabemos se foi lentidão ou queda”	Classificação de cada check e saúde degradado/indisponível	`domain/outcomes.py`, `domain/health.py`
“Um erro de rede isolado vira alarme”	Limiares de falhas e sucessos seguidos	`domain/health.py`
“Não sabemos por que o sistema decidiu isso”	Motivo gravado em cada mudança de estado e no incidente	`health_changes`, `incident_events`
“Recebemos o mesmo alerta dez vezes”	Só transições geram alerta; identificador único do evento	`services/alerts.py`
“Se o canal de alerta cai, perdemos o aviso”	Outbox com retentativas	`services/alerts.py`, laço de entregas
“Um alerta travado atrasa tudo”	Entregas em laço separado dos checks	`worker/runner.py`
“Quanto tempo ficou fora do ar este mês?”	Métricas por janela, com N/A quando não há dados	`services/queries.py`, `domain/metrics.py`
“Alguém pode usar o monitor para atacar a rede interna?”	Política SSRF	`security/`
“Como mostramos isso funcionando?”	Laboratório de falhas	`lab/`, `scripts/demo.sh`

1.6 Cenário fictício e limites do projeto

O cenário é **fictício**: não existe a equipe, nem as APIs dela. Ele serve para dar direção aos requisitos. O PulseOps é um projeto pessoal de estudo, não um produto em uso.

Limites assumidos (detalhados no README e no modelo de ameaças):

- um operador só, sem cadastro de usuários e papéis;
- um worker só, sem escala horizontal;
- só HTTP/HTTPS, sem TCP, ping ou DNS;

- sem autenticação nos alvos (headers secretos ficam no roadmap);
- webhooks sem assinatura HMAC;
- sem limite de tentativas de login;
- quando a máquina inteira cai, o PulseOps não consegue avisar ninguém.

Funcionalidades comuns, como checar HTTP, mandar alerta e mostrar um painel, não são invenções do projeto. Ferramentas maduras fazem isso há anos (parte I mostra como falar disso numa entrevista). O que o PulseOps tenta mostrar é a lógica bem resolvida: regra explícita, histórico confiável, alertas desacoplados e testes que provam cada comportamento.

2 Parte B — Execução passo a passo

Esta parte leva do zero até o painel funcionando. Para cada comando há três coisas: o que cada pedaço significa, o que deve aparecer e como confirmar que deu certo.

> **Onde isto foi testado.** Os comandos foram executados em Linux (container x86_64 com Docker 29). Em macOS devem funcionar igual. Em Windows, use o WSL2 (Ubuntu) com o Docker Desktop: os scripts `.sh` precisam de um shell bash. Windows sem WSL **não foi testado**.

2.1 Terminal e diretórios

O **terminal** é um programa onde você digita comandos em texto. Ele sempre está "dentro" de uma pasta, o **diretório atual**.

Comando	O que faz
`pwd`	mostra o diretório atual (*print working directory*)
`ls`	lista os arquivos do diretório atual; `ls -a` inclui os ocultos, que começam com ponto, como `.env`
`cd pulseops`	entra na pasta `pulseops`; `cd ..` volta uma pasta
`cat arquivo`	mostra o conteúdo de um arquivo

Caminhos começando com `/` são **absolutos** (a partir da raiz do disco). Os outros são **relativos** ao diretório atual. `./scripts/demo.sh` e `scripts/demo.sh` são o mesmo arquivo quando você está na raiz do projeto.

2.2 Instalação das ferramentas

1. **Git**, para baixar o código: `git --version` deve mostrar algo como `git version 2.4x`.
2. **Docker** com Compose v2: `docker --version` e `docker compose version`. O segundo comando precisa responder `Docker Compose version v2...` (com espaço, não `docker-compose` com hífen, que é a versão antiga).
3. Para rodar os testes fora do Docker: **uv** (`curl -LsSf https://astral.sh/uv/install.sh | sh`) e conferir com `uv --version`.

2.3 Baixar o código

```
git clone <url-do-repositorio> pulseops
cd pulseops
```

- `git clone` copia o repositório inteiro, com o histórico, para uma pasta nova.
- `<url-do-repositorio>` é o endereço que aparece no botão "Code" do GitHub.
- `pulseops` no fim é o nome da pasta criada.
- **Confirmação:** `ls` mostra `README.md`, `compose.yaml`, `src`, `tests`...

2.4 Ambientes e dependências

O código Python depende de bibliotecas de terceiros (FastAPI, SQLAlchemy...). As versões exatas ficam em dois arquivos:

- `pyproject.toml`: a lista do que o projeto precisa, com versões mínimas.
- `uv.lock`: a versão **exata** de cada pacote, inclusive das dependências das dependências. Com ele, a instalação de hoje e a de daqui a um ano resultam no mesmo conjunto.

Um **ambiente virtual** (`.venv/`) é uma pasta com um Python e as bibliotecas do projeto, separada do Python do sistema. Assim, projetos diferentes não brigam por versões.

```
uv sync
```

- `uv` é o gerenciador de pacotes escolhido (ADR 0001).
- `sync` cria o `.venv/` se não existir e instala exatamente o que está no `uv.lock`, incluindo as ferramentas de desenvolvimento (pytest, ruff, mypy).
- **Confirmação:** `uv run python -c "import pulseops; print(pulseops.__version__)"` imprime `0.1.0`.

`uv run <comando>` executa o comando **dentro** do ambiente virtual, sem precisar "ativá-lo".

Para rodar o PulseOps com Docker, nada disso é necessário: a imagem instala as dependências sozinha. O `uv` só é preciso para testes e desenvolvimento.

2.5 Variáveis de ambiente

Uma **variável de ambiente** é um par nome=valor que o sistema entrega a um programa quando ele começa. O PulseOps lê todas as que começam com `PULSEOPS_` (arquivo `src/pulseops/config.py`). Isso permite mudar a configuração sem mudar o código, e deixa senhas fora do Git.

```
cp .env.example .env
```

- `cp` copia `.env.example` (modelo, que vai para o Git) para `.env` (seu, que **não** vai: está no `.gitignore`).
- Abra o `.env` num editor e troque `POSTGRES_PASSWORD=troque-esta-senha` por uma senha sua.
- O Docker Compose lê o `.env` automaticamente e repassa os valores.

As variáveis mais úteis estão em `docs/operacao.md`. Exemplo: com `PULSEOPS_DISPLAY_TIMEZONE=America/Sao_Paulo`, o painel mostra horários de Brasília; o banco continua guardando tudo em UTC.

2.6 Docker e Compose

- Uma **imagem** é um pacote com sistema operacional mínimo, Python, bibliotecas e o código. O `Dockerfile` é a receita da imagem.
- Um **container** é uma imagem em execução, isolada do resto da máquina.

- O **Docker Compose** sobe vários containers juntos a partir do `compose.yaml`. No PulseOps são quatro serviços: `db` (PostgreSQL), `migrate` (cria as tabelas e sai), `api` (painel e API) e `worker` (checks e alertas).
- Um **volume** (`pgdata`) é um disco que sobrevive aos containers: apagar o container do banco não apaga os dados.

```
docker compose up -d --build --wait
```

Pedaço	Significado
`docker compose`	usa o Compose com o `compose.yaml` da pasta atual
`up`	cria e inicia os serviços
`-d`	*detached*: roda em segundo plano e devolve o terminal
`--build`	(re)constrói a imagem a partir do `Dockerfile` antes de subir
`--wait`	espera os serviços ficarem saudáveis (healthchecks) antes de terminar

Resultado esperado: no fim, linhas como `Container pulseops-api-1 Healthy`. A primeira vez demora alguns minutos (download das imagens base e das bibliotecas).

Confirmação:

```
docker compose ps -a
```

`db` e `api` aparecem `running (healthy)`, `worker` aparece `running` e `migrate` aparece `exited (0)`. O 0 é o código de saída: terminou sem erro, como esperado.

2.7 Banco e migrações

O banco começa vazio. Uma **migração** é um script que cria ou altera tabelas. O PulseOps usa o **Alembic**, e a única migração está em `migrations/versions/0001_esquema_inicial.py`. O serviço `migrate` executa `alembic upgrade head` ("aplique todas as migrações até a mais recente") toda vez que o ambiente sobe; se já estiverem aplicadas, ele não faz nada.

Para ver as tabelas criadas:

```
docker compose exec db psql -U pulseops -d pulseops -c '\dt'
```

- `exec db` roda um comando **dentro** do container `db` que já está no ar.
- `psql` é o cliente de linha de comando do PostgreSQL; `-U` é o usuário e `-d`, o banco.
- `\dt` lista as tabelas. Devem aparecer 13 tabelas do PulseOps (`monitors`, `checks`, `incidents`...) e a `alembic_version`, onde o Alembic anota qual migração foi aplicada.

2.8 Criar a operadora

```
docker compose run --rm api pulseops create-admin --username amanda
```

- `run` cria um container **temporário** a partir do serviço `api` só para rodar um comando. `--rm` o apaga no fim.
- `pulseops create-admin` é o comando definido em `src/pulseops/cli.py`.
- O comando pergunta a senha duas vezes, sem mostrar o que você digita. Mínimo de 12 caracteres.
- **Resultado esperado:** `operador 'amanda' criado`.

2.9 Iniciar, usar e parar

1. Abra http://127.0.0.1:8000. Você será levada para `/login`.
2. Entre com o usuário criado. O painel mostra "Nenhum monitor cadastrado".
3. Clique em **Novo monitor**, informe nome e URL pública (por exemplo, `https://example.com/`) e salve. Fora do modo demonstração, o intervalo mínimo é 30 s.
4. Em poucos segundos, a página do monitor mostra o primeiro check.

`127.0.0.1` é o "localhost", a própria máquina. O Compose publica a porta 8000 **só** nesse endereço, então outras máquinas da rede não acessam o painel.

```
docker compose stop      # para os containers (mantém tudo)
docker compose start     # volta a subir
docker compose down      # remove os containers e a rede; o volume fica
docker compose down -v   # remove também o volume: APAGA o banco
```

2.10 Como conferir se está funcionando

O que conferir	Comando	Resultado esperado
API viva	`curl -s http://127.0.0.1:8000/health/live`	`{"status":"ok"}`
API com banco	`curl -s http://127.0.0.1:8000/health/ready`	`"status":"ok"` e `"database":"ok"`
Worker vivo	painel inicial	nenhum aviso "Worker sem sinal"
Checks acontecendo	página do monitor	"Histórico de checks" crescendo a cada intervalo
Logs do worker	`docker compose logs -f worker`	linhas JSON com `"msg": "check gravado"`

`curl` é um cliente HTTP de linha de comando; `-s` (*silent*) esconde a barra de progresso. `logs -f` (*follow*) continua mostrando linhas novas até você apertar `Ctrl+C`.

2.11 O laboratório de falhas

```
scripts/demo.sh
```

Sobe o ambiente com `compose.yaml` **e** `compose.demo.yaml`, ativando o perfil `demo` (alvo e receptor controláveis, intervalos curtos), cria a operadora `demo` e executa os 11 passos descritos no README. A senha da operadora `demo` fica em `.demo/admin-password`. Para derrubar: `scripts/demo.sh --down`.

Confirmação: a última linha é `Demonstração concluída: os 11 passos foram verificados.` e `echo $?` logo depois mostra `0`. Se um passo falhar, aparece `FALHOU: <motivo>` e o código é 1.

2.12 Rodar os testes

```
docker run -d --name pulseops-testdb -e POSTGRES_USER=pulseops \
  -e POSTGRES_PASSWORD=pulseops -p 127.0.0.1:5433:5432 postgres:16-alpine
uv run pytest
```

- O primeiro comando cria um PostgreSQL só para testes na porta 5433 (a barra invertida `\` no fim da linha continua o comando na linha seguinte).
- `-e` define variáveis de ambiente do container; `-p 127.0.0.1:5433:5432` liga a porta 5433 da sua máquina à 5432 do container.
- `uv run pytest` encontra e executa todos os testes em `tests/`.
- **Resultado esperado:** `251 passed` (o número exato do commit explicado), em menos de um minuto.

Os testes apagam e recriam o banco `pulseops_test` a cada execução. Nunca aponte `PULSEOPS_TEST_DATABASE_URL` para um banco com dados que importam.

2.13 Erros comuns e diagnóstico

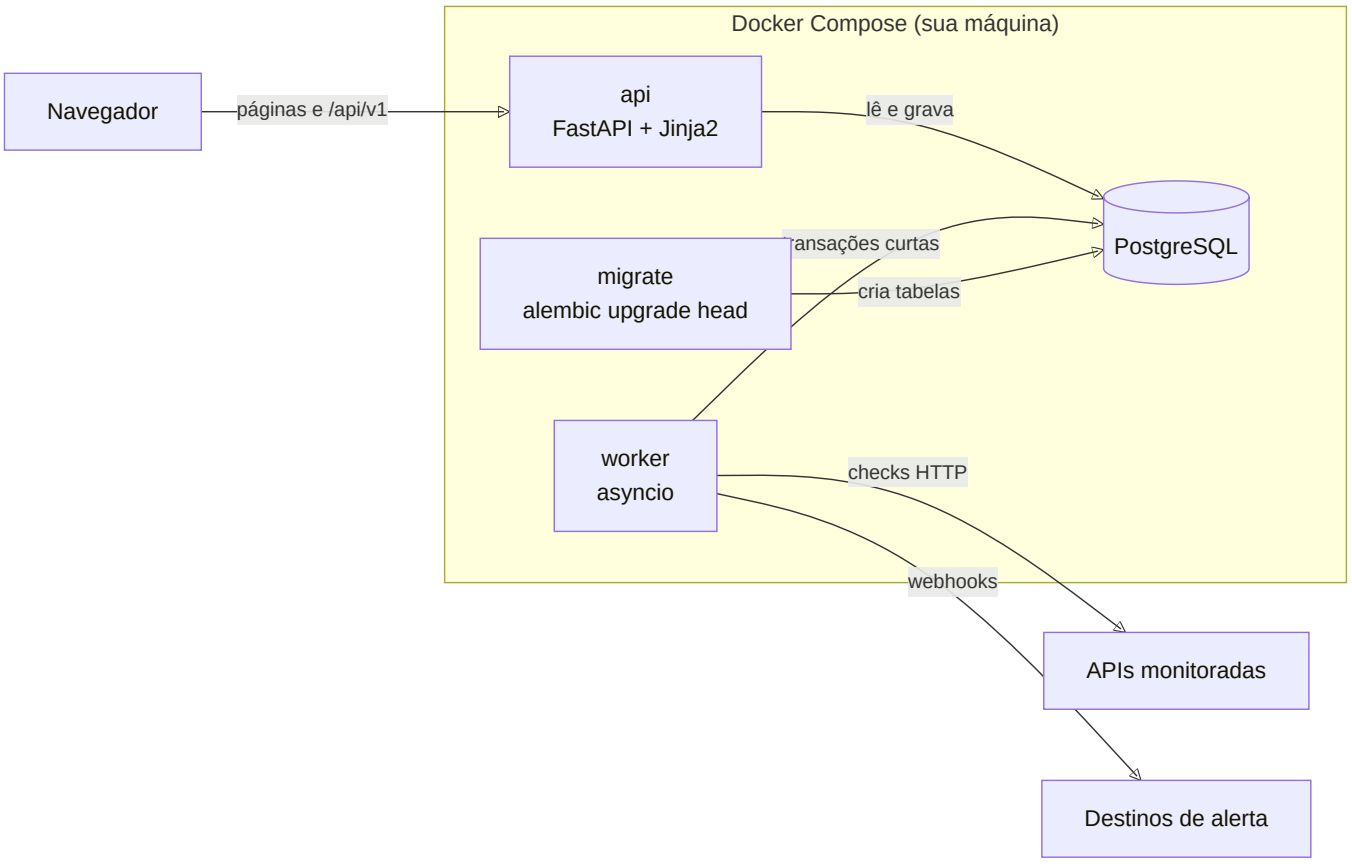
Sintoma	Causa provável	Como diagnosticar e resolver
`port is already allocated` ao subir	Outro programa usa a porta 8000	Defina `PULSEOPS_PORT=8001` no `.env` e acesse `:8001`
`Cannot connect to the Docker daemon`	Docker parado	Abra o Docker Desktop ou `sudo systemctl start docker`
`toomanyrequests` ao baixar imagens	Limite de downloads do Docker Hub	Espere alguns minutos ou faça login com `docker login`
Erro de certificado no build	Rede com proxy TLS corporativo	`PULSEOPS_EXTRA_CA_FILE=/caminho/ca.pem` antes do `docker compose up`
`migrate` com `exited (1)`	Banco inacessível ou senha errada	`docker compose logs migrate`; confira `POSTGRES_PASSWORD`. Se você trocou a senha **depois** de criar o volume, o banco antigo continua com a senha antiga: use a antiga ou apague o volume (`down -v`, perde os dados)
Monitor recusado: "endereço privado"	Política SSRF, de propósito	Monitores só alcançam endereços públicos. Para testes locais, use o laboratório

Sintoma	Causa provável	Como diagnosticar e resolver
Checks com “falha de DNS”	O container não resolve nomes	`docker compose exec worker python -c "import socket; print(socket.getaddrinfo('example.com', 443))"`
Todos os monitores “desatualizados”	Worker parado	`docker compose ps worker` e `docker compose logs worker`
`pytest` com `connection refused` na porta 5433	Banco de testes não está rodando	`docker start pulseops-testdb`

Quando algo não funcionar, a ordem de investigação é: `docker compose ps -a` (o que está rodando?), `docker compose logs <serviço>` (o que ele disse?) e `curl .../health/ready` (a API alcança o banco?).

3 Parte C — Arquitetura

3.1 Os processos



O PulseOps roda como **dois processos Python** que não conversam diretamente: os dois leem e gravam no mesmo PostgreSQL.

- **api**: o servidor web (uvicorn + FastAPI). Responde ao navegador e à API JSON. Nunca faz checks nem envia webhooks.
- **worker**: um programa que fica em laço. Ele reserva checks vencidos, faz as requisições, grava os resultados, entrega alertas e faz manutenção.

Por que separar? Se o agendador rodasse dentro do servidor web, cada processo do uvicorn teria o próprio agendador, e os checks rodariam em dobro. Um servidor web também pode ser reiniciado a qualquer momento; um laço preso nele morreria junto (ADR 0002). Como a fila de trabalho mora no banco (`monitors.next_run_at` e a tabela `deliveries`), a API só precisa gravar e o worker só precisa ler.

3.2 Responsabilidade de cada pasta e arquivo

3.2.1 Raiz do repositório

Arquivo	Responsabilidade
`pyproject.toml`	Metadados do pacote, dependências, configuração de Ruff, mypy, pytest e cobertura
`uv.lock`	Versões exatas de todas as dependências (gerado pelo uv)
`Dockerfile`	Receita da imagem usada por api, worker, migrate e laboratório
`.dockerignore`	O que não entra no contexto de build da imagem
`compose.yaml`	Serviços do ambiente normal: db, migrate, api, worker
`compose.demo.yaml`	Acréscimos do laboratório: modo demo, alvo, receptor e roteiro
`alembic.ini`	Configuração do Alembic (onde estão as migrações)
`.env.example`	Modelo de variáveis de ambiente, sem segredos reais
`docker/no-extra-ca.pem`	Arquivo vazio usado quando não há certificado extra no build
`.github/workflows/ci.yml`	Integração contínua no GitHub Actions
`README.md`, `LICENSE`, `CLAUDE.md`	Apresentação, licença MIT, instruções para retomar o trabalho

3.2.2 `src/pulseops/` — a aplicação

Arquivo	Responsabilidade
`config.py`	`Settings`: lê as variáveis `PULSEOPS_*` e valida
`clock.py`	`Clock` (protocolo) e `SystemClock`: a fonte de "agora" substituível nos testes
`logs.py`	Logs em JSON com contexto e mascaramento de dados sensíveis
`cli.py`	Comando `pulseops create-admin`
`domain/outcomes.py`	`Outcome`: os oito resultados possíveis de um check
`domain/status_codes.py`	Interpreta `"200-299,301"` e responde se um status é aceito
`domain/monitor_rules.py`	`MonitorConfig` e `validate_config`: limites e regras combinadas
`domain/health.py`	`HealthTracker`: a regra de saúde e de incidentes (o coração do projeto)
`domain/metrics.py`	Fórmulas puras: nearest-rank, razão segura, duração recortada pela janela

Arquivo	Responsabilidade
`security/url_policy.py`	`DestinationPolicy`: quais URLs e IPs são permitidos
`security/transport.py`	Cliente HTTPX que resolve, valida e conecta no IP validado
`security/passwords.py`	Hash e verificação de senha com Argon2id
`checks/checker.py`	`HttpChecker`: faz um check e classifica o resultado
`notifications/webhook.py`	`WebhookNotifier`: envia um evento por POST
`db/models.py`	As 13 tabelas como classes do SQLAlchemy
`db/session.py`	Criação da conexão (engine) e da fábrica de sessões
`services/errors.py`	Exceções dos casos de uso (não encontrado, conflito, validação)
`services/monitors.py`	Criar, editar, pausar, retomar, arquivar, verificar agora
`services/processing.py`	Reservar execuções, gravar resultados, manutenção
`services/incidents.py`	Abrir, atualizar, resolver e encerrar incidentes
`services/alerts.py`	Outbox: criar eventos e entregas, reservar, registrar tentativas
`services/auth.py`	Operadores e sessões de login
`services/queries.py`	Consultas de leitura: listas, resumo, histórico, métricas
`worker/runner.py`	`Worker`: os três laços (checks, entregas, manutenção)
`worker/__main__.py`	Ponto de entrada do worker: trava única, sinais, clientes HTTP
`web/app.py`	`create_app`: monta o FastAPI, middlewares e tratamento de erros
`web/deps.py`	Dependências injetadas nas rotas: estado, sessão do banco, autenticação
`web/schemas.py`	Modelos Pydantic de entrada e saída da API
`web/api.py`	Rotas JSON em `/api/v1`
`web/pages.py`	Rotas das páginas HTML e login/logout
`web/health.py`	`/health/live` e `/health/ready`
`web/templating.py`	Ambiente Jinja2 com autoescape e filtros de formatação
`web/charts.py`	Monta o gráfico de latência em SVG
`web/templates/*.html`	As páginas
`web/static/style.css`, `app.js`	Estilo e o único JavaScript (atualiza o resumo)

3.2.3 Laboratório, scripts e testes

Caminho	Responsabilidade
`lab/target.py`	Alvo controlável: saudável, lento, erro 500, timeout

Caminho	Responsabilidade
`lab/receiver.py`	Receptor de webhooks com deduplicação e modo indisponível
`scripts/demo.sh`, `scripts/demo_flow.py`	Sobem o laboratório e executam os 11 passos
`scripts/ensaio_carga.sh`, `scripts/ensaio_carga.py`	Ensaio de carga local
`scripts/conferir_referencias.py`	Confere se testes citados na documentação existem
`scripts/capturar_telas.mjs`	Gera as capturas de tela (opcional, usa Node)
`migrations/env.py`, `migrations/versions/0001_esquema_inicial.py`	Configuração do Alembic e a migração
`tests/unit/`	Testes sem banco
`tests/integration/`	Testes com PostgreSQL real
`tests/conftest.py`, `tests/support.py`, `tests/support_db.py`	Fixtures e dublês compartilhados

3.3 Rota, schema, entidade e serviço

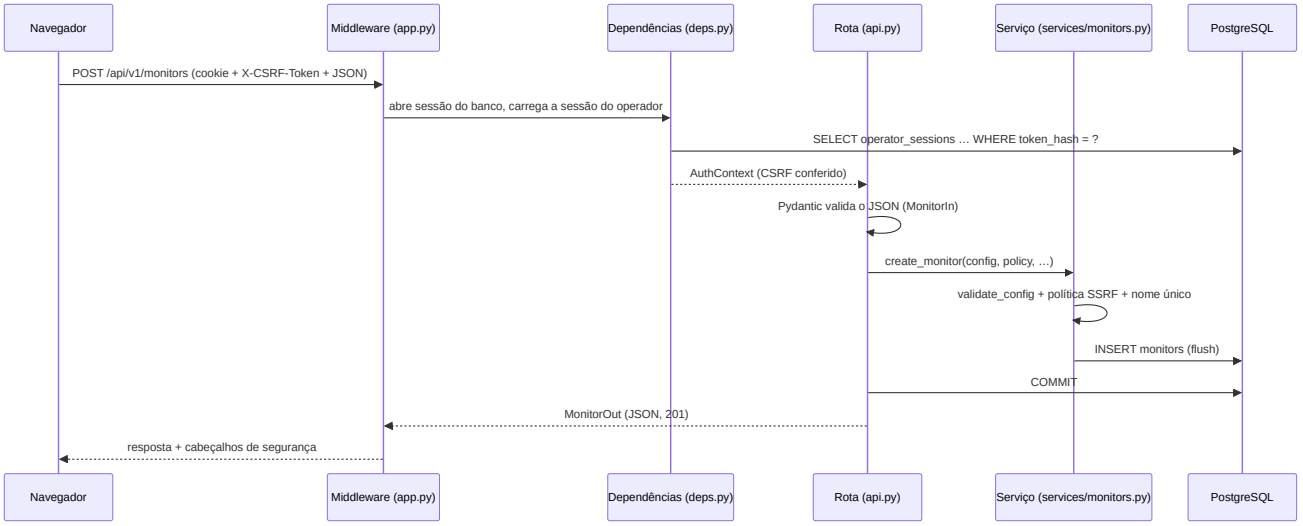
Esses quatro nomes aparecem o tempo todo. Usando o cadastro de monitor como exemplo:

Peça	O que é	Exemplo no PulseOps	Sabe de HTTP?	Sabe de banco?
Rota	Função ligada a um método e caminho HTTP; recebe a requisição e devolve a resposta	`create_monitor` em `web/api.py`, ligada a `POST /api/v1/monitors`	sim	só repassa a sessão
Schema	Classe Pydantic que descreve o formato de entrada ou saída e valida tipos	`MonitorIn` e `MonitorOut` em `web/schemas.py`	descreve o JSON	não
Entidade (modelo)	Classe SQLAlchemy que corresponde a uma tabela	`Monitor` em `db/models.py`	não	sim
Serviço (caso de uso)	Função que executa a regra de negócio, usando entidades e regras de domínio	`create_monitor` em `services/monitors.py`	não	sim

E uma quinta peça, a **regra de domínio**: código puro, sem HTTP nem banco, como `validate_config` e `HealthTracker`. É a parte mais fácil de testar.

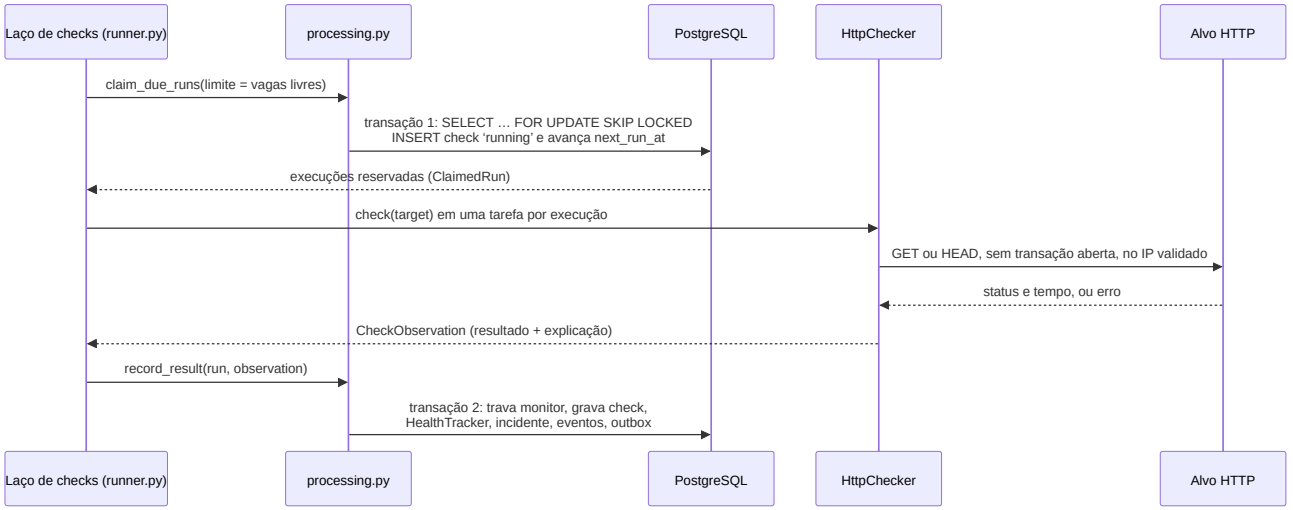
Por que não colocar tudo na rota? Porque a mesma regra é usada por caminhos diferentes: o cadastro pela API JSON (`web/api.py`) e pelo formulário HTML (`web/pages.py`) chamam o **mesmo** `services/monitors.create_monitor`. Se a regra estivesse dentro da rota, existiriam duas cópias, e um dia elas divergiriam.

3.4 Caminho de uma requisição



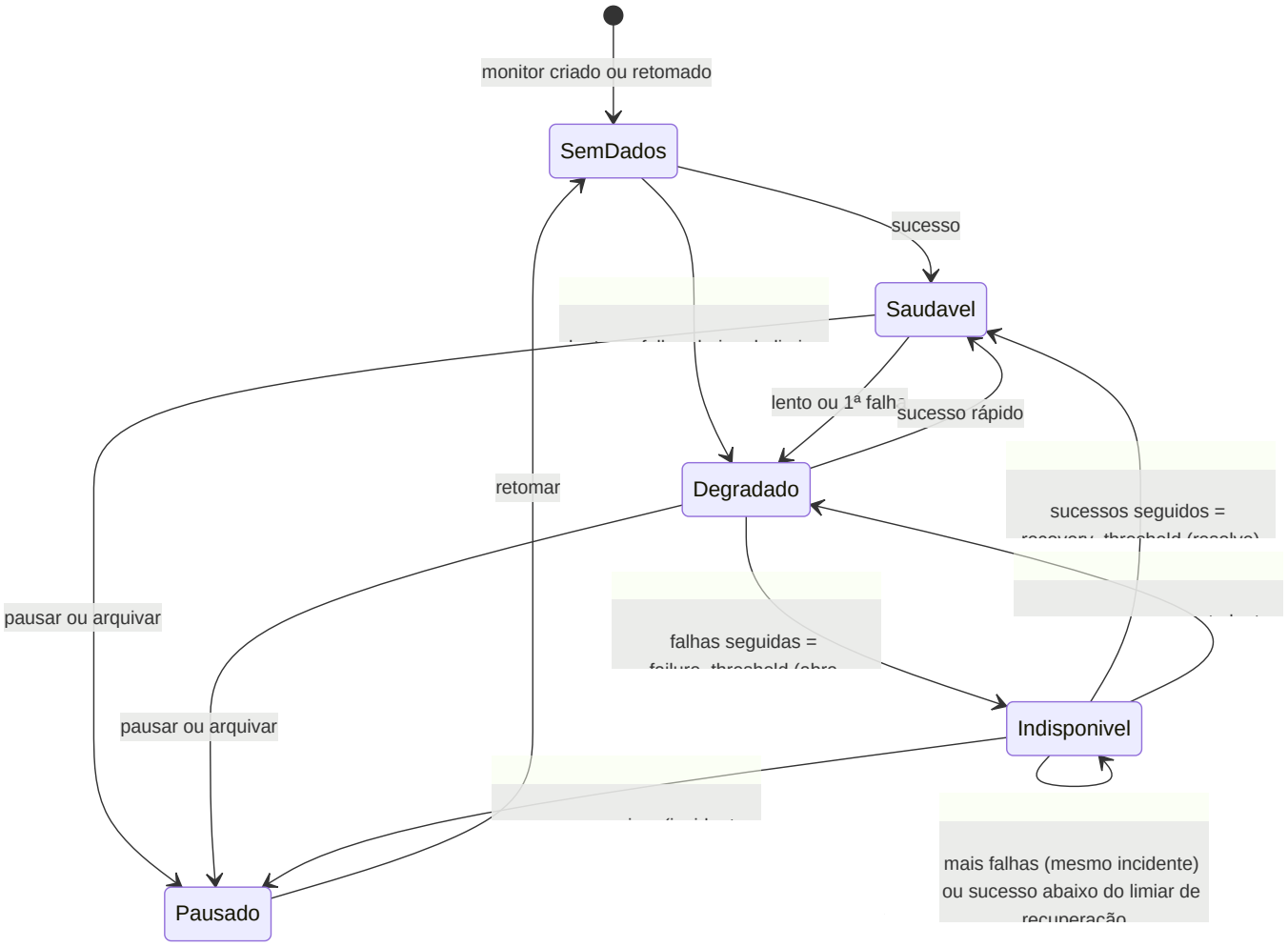
1. O navegador envia `POST /api/v1/monitors` com o cookie de sessão, o cabeçalho `X-CSRF-Token` e o JSON.
2. O middleware de `web/app.py` envolve tudo: no fim, acrescenta cabeçalhos de segurança (CSP, `X-Frame-Options`…).
3. O FastAPI resolve as **dependências** declaradas na assinatura da rota (`db: Db, state: State, _: ApiAuth`): abre uma sessão do banco, carrega a sessão de login pelo hash do cookie e confere o token CSRF. Sem sessão, 401; sem CSRF válido, 403.
4. O Pydantic converte o JSON em `MonitorIn`. Campo desconhecido ou tipo errado gera 422 antes de a rota rodar.
5. A rota chama o serviço, que valida as regras combinadas, aplica a política SSRF, confere nome repetido e faz `INSERT` (com `flush`, ainda sem confirmar).
6. A rota faz `commit` e devolve `MonitorOut` com status 201.
7. Erros de regra viram respostas HTTP: `ValidationFailedError` → 422, `NotFoundError` → 404, `ConflictError` → 409.

3.5 Caminho de uma verificação



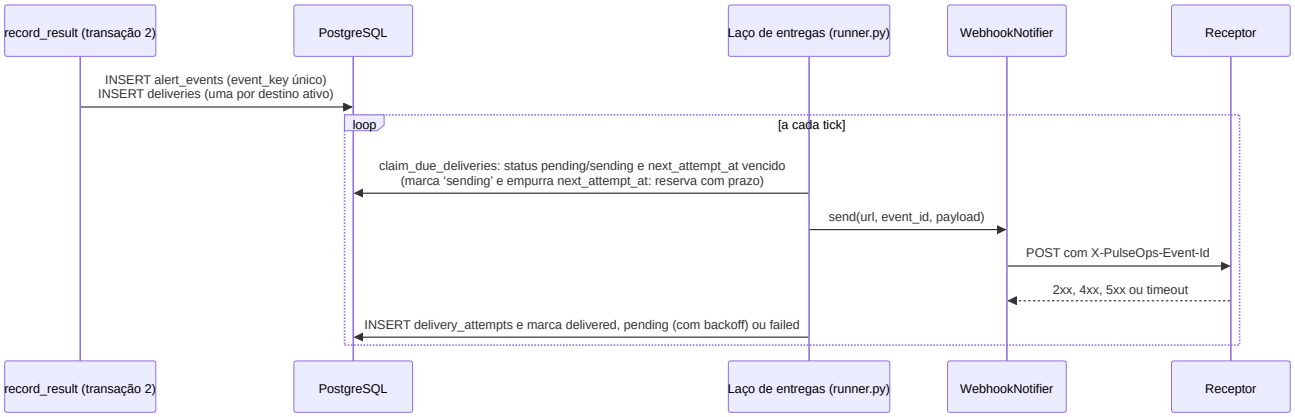
1. O laço de checks do worker (`Worker._check_loop`) calcula as vagas livres (`worker_concurrency` menos os checks em andamento).
2. `claim_due_runs` abre uma transação curta: seleciona monitores ativos com `next_run_at` vencido e sem check em andamento, travando as linhas (`FOR UPDATE SKIP LOCKED`). Para cada um, insere um check `running` com `run_key = "<id>:<horário previsto>"` e avança `next_run_at` em um intervalo. Confirma e libera as travas.
3. Para cada execução reservada, o worker cria uma tarefa `asyncio` que chama `HttpChecker.check`. A requisição acontece **sem transação aberta**: um alvo que demora 30 s não segura nada no banco.
4. O transporte protegido resolve o nome, valida todos os IPs e conecta no IP validado. O verificador mede o tempo, lê só o status (nunca o corpo) e classifica o resultado em um `Outcome` com uma explicação curta.
5. `record_result` abre a segunda transação: trava o monitor, grava o resultado no check e, se o monitor continua ativo e com a mesma `config_version`, chama `_apply_domain`.
6. `_apply_domain` monta o estado atual (`TrackerState`), pergunta ao `HealthTracker` o que fazer e aplica a decisão: contadores, saúde, mudança de estado com motivo e, se for o caso, incidente e alerta. Tudo na mesma transação: ou tudo é gravado, ou nada.
7. Quando a tarefa termina, ela libera a vaga e acorda o laço, que reserva a próxima execução se houver fila (correção do bug B8).

3.6 Caminho de um incidente



- **Abertura** (`services/incidents.open_incident`): cria o incidente com `started_at` = primeira falha e `confirmed_at` = check atual, copia os limiares vigentes, grava dois eventos na linha do tempo (`first_failure` e `opened`) e enfileira o alerta `incident.opened`. O `flush` força o banco a conferir o índice único parcial: se já houvesse um incidente aberto, a transação inteira falharia.
- **Atualização** (`record_failure_on_open_incident`): falhas seguintes só somam `failure_count` e trocam o último check. Se o **tipo** de falha mudar (de timeout para erro 500, por exemplo), entra um evento `error_changed`. Nenhum alerta novo.
- **Recuperação** (`resolve_incident`): status `resolved`, `ended_at`, motivo, evento `resolved` e alerta `incident.resolved`.
- **Encerramento pelo operador** (`close_incident_by_operator`): pausar ou arquivar com incidente aberto encerra como `closed`, com motivo explícito de que não houve recuperação observada, e alerta `incident.closed` (ADR 0006).
- **Mudança de configuração** (`record_config_change`): o incidente continua aberto; a linha do tempo registra o que mudou e quantos sucessos passam a ser exigidos.

3.7 Caminho de uma notificação



1. Na mesma transação que muda o incidente, `enqueue_incident_alert` grava o evento em `alert_events` (com `event_key` único, por exemplo `incident:12:opened`) e uma linha em `deliveries` para cada destino ativo. Esse é o padrão **outbox**: o alerta não é enviado agora, ele é registrado.
2. O laço de entregas, independente do laço de checks, chama `claim_due_deliveries`: pega entregas `pending` vencidas (ou `sending` cuja reserva expirou), marca como `sending` e empurra `next_attempt_at` para frente (uma "reserva com prazo").
3. `WebhookNotifier.send` faz o POST com o cabeçalho `X-PulseOps-Event-Id`, com timeout próprio e a mesma política SSRF dos checks.
4. `record_delivery_result` grava a tentativa e decide: 2xx → `delivered`; 4xx (exceto 408 e 429) → `failed` (não adianta repetir); outros erros → `pending` com espera `min(base × 2^(n−1), teto)`; tentativas esgotadas → `failed`.
5. Se o worker cair depois de enviar e antes de gravar, a reserva expira e a entrega é tentada de novo. Por isso a entrega é "pelo menos uma vez" e o receptor deve ignorar ids repetidos.

3.8 Por que nenhuma fila externa

Não há Redis, Celery nem Kafka. A "fila" de checks é a coluna `next_run_at` com o índice `ix_monitors_due`; a de alertas é a tabela `deliveries` com o índice `ix_deliveries_due`. Para um worker e algumas centenas de monitores, o PostgreSQL dá conta (o ensaio de carga mediu 200 monitores a cada 10 s) e as garantias de consistência ficam em um só lugar. Uma fila externa entraria se houvesse muitos workers ou volume muito maior, e isso exigiria medir antes (parte H, pergunta sobre escala).

4 Parte E — Fundamentos de Python aplicados

Cada seção explica um conceito e mostra **onde ele aparece no PulseOps e por quê**. Quando um conceito é citado só para comparação, isso está dito.

4.1 Tipos

Todo valor em Python tem um tipo: `int` (inteiro), `float` (número com casas decimais), `str` (texto), `bool` (`True`/`False`), `None` (ausência de valor), `datetime` (data e hora). No PulseOps:

- `interval_seconds` é `int`; `worker_tick_seconds` é `float`, porque pode ser 0,5 s.
- Datas são sempre `datetime` **com fuso** (*timezone-aware*), em UTC. Um `datetime` sem fuso é ambíguo ("10h de onde?"); a API recusa datas assim (`_require_aware` em `web/api.py`) e o banco usa `timestamptz`.
- `None` aparece onde algo pode não existir: `Monitor.next_run_at` é `None` quando o monitor está pausado; `Check.status_code` é `None` quando não houve resposta.

Enum é um tipo com um conjunto fechado de valores. `Outcome` herda de `StrEnum`, então cada membro também é uma `str` e pode ser gravado direto no banco:

`src/pulseops/domain/outcomes.py, linhas 6–28`

```
 6 class Outcome(StrEnum):
 7     HEALTHY = "healthy"
 8     SLOW = "slow"
 9     UNEXPECTED_STATUS = "unexpected_status"
10     TIMEOUT = "timeout"
11     CONNECTION_ERROR = "connection_error"
12     TLS_ERROR = "tls_error"
13     BLOCKED = "blocked"
14     INTERNAL_ERROR = "internal_error"
15
16     @property
17     def is_failure(self) -> bool:
18         """Falha do serviço monitorado (RF03.1). Erro interno do PulseOps não é falha do alvo."""
19         return self in _FAILURES
20
21     @property
22     def is_success(self) -> bool:
23         """Respondeu com status aceito, rápido ou lento (RF03.2)."""
24         return self in (Outcome.HEALTHY, Outcome.SLOW)
25
26     @property
27     def label(self) -> str:
28         return _LABELS[self]
```

`Outcome.TIMEOUT == "timeout"` é verdadeiro. As `@property` (`is_failure`, `is_success`, `label`) colocam o conhecimento sobre cada resultado junto do tipo, em vez de espalhar `if outcome in (...)` pelo código.

4.2 Coleções

Coleção	Característica	Onde aparece
`list`	ordenada, mutável, aceita repetidos	`ClaimedRun`s reservados em `claim_due_runs`; checks de uma página
`tuple`	ordenada, imutável	pares `(host, porta)` da lista do laboratório; `ANSWERED = ("healthy", "slow")` em `charts.py`
`dict`	pares chave → valor	`_LABELS` em `outcomes.py`; `payload` do webhook; contagem `by_outcome` nas métricas
`set`	sem ordem, sem repetidos, busca rápida	`Worker._inflight`, as tarefas em andamento
`frozenset`	`set` imutável	`_FAILURES` em `outcomes.py`; `UNSAFE_METHODS` em `web/deps.py`; portas permitidas

Por que `frozenset` para `_FAILURES`? Porque o conjunto de falhas é uma constante: imutável, ninguém o altera por engano, e `x in conjunto` é rápido.

Compreensões criam coleções a partir de outras numa linha. Exemplo de `config.py`: `frozenset(int(p) for p in self.allowed_ports.split(",") if p.strip())` transforma `"80,443"` em `frozenset({80, 443})`, ignorando pedaços vazios. A correção do bug B2 trocou `dict(resultado)` por uma compreensão `{status: count for status, count in rows}` (parte G).

4.3 Funções e argumentos

- **Argumentos só por nome.** O `*` na assinatura obriga a nomear o que vem depois: `claim_due_runs(sessions, *, now, limit)` só pode ser chamada como `claim_due_runs(sessions, now=..., limit=...)`. Em funções com vários valores do mesmo tipo (dois `datetime`, dois `int`), isso evita trocar a ordem sem perceber. O PulseOps usa esse padrão em quase todos os serviços.
- **Valores padrão**: `create_monitor(..., resolver: Resolver | None = None)`.
- `**fields` junta argumentos nomeados num `dict`: `log_context(monitor_id=1, run_key="...")` em `logs.py`.
- **Funções como valores**: em `web/api.py`, `operations = {"pause": monitor_service.pause_monitor, ...}` guarda funções num dicionário e escolhe qual chamar pelo nome da ação.

4.4 Imports e módulos

Cada arquivo `.py` é um **módulo**; cada pasta com `__init__.py` é um **pacote**. `from pulseops.domain.health import HealthTracker` busca a classe no arquivo `src/pulseops/domain/health.py`. O projeto usa o layout `src/`: o código fica em `src/pulseops/` e só é

importável depois de instalado (`uv sync` instala o próprio projeto). Isso evita importar sem querer arquivos soltos da pasta atual.

Os imports seguem uma direção: `web` e `worker` importam `services`; `services` importa `domain`, `db` e `security`; `domain` não importa nada do projeto. Essa direção é o que torna o domínio testável sem banco.

4.5 Classes e instâncias

Uma **classe** é um molde; uma **instância** é um objeto criado a partir dele. `HealthTracker(Thresholds(3, 2))` cria uma instância que guarda os limiares em `self._thresholds`. O `self` é a própria instância, passado automaticamente.

Dataclasses geram `__init__`, comparação e representação a partir das anotações. `@dataclass(frozen=True)` torna a instância imutável:

`src/pulseops/domain/health.py, linhas 46–62`

```python
@dataclass(frozen=True)
class TrackerState:
    """Parte do monitor que a regra lê e altera. Fica persistida no banco."""

    health: Health
    consecutive_failures: int
    consecutive_successes: int
    first_failure_at: datetime | None
    incident_open: bool


@dataclass(frozen=True)
class Observation:
    outcome: Outcome
    observed_at: datetime
    explanation: str
```

`TrackerState` é imutável de propósito: o `HealthTracker` nunca altera o estado recebido; ele devolve um estado novo com `dataclasses.replace(...)`. Assim fica impossível um teste "contaminar" o estado de outro, e a função é fácil de entender: entra estado e observação, sai decisão.

4.6 Encapsulamento e composição

Encapsulamento é esconder os detalhes atrás de uma interface pequena. O `HealthTracker` expõe só `apply(state, observation)`; as regras ficam em métodos privados (`_on_failure`, `_on_success`, com `_` indicando "uso interno"). Quem chama não precisa saber como a contagem funciona.

Composição é montar um objeto a partir de outros, em vez de herdar. Exemplos:

- `HealthTracker` **tem** um `Thresholds`.
- `Worker` **tem** um `Checker`, um `Notifier`, um `Clock` e uma fábrica de sessões, recebidos no construtor.
- `GuardedNetworkBackend` **tem** uma `DestinationPolicy`, um `Resolver` e o backend de rede real, a quem repassa a conexão depois de validar.

A alternativa seria herança, por exemplo um `WorkerDeTeste(Worker)` que sobrescrevesse o método de fazer requisição. Com composição, o teste só entrega outro `Checker` ao mesmo `Worker`. Nada de herança foi necessário no código de domínio; ela aparece só onde a biblioteca exige (`Base(DeclarativeBase)` no SQLAlchemy, `BaseModel` no Pydantic, `BaseSettings`).

4.7 Protocolos (interfaces)

Um `Protocol` descreve **o que** um objeto precisa ter, sem dizer de qual classe ele vem. Qualquer classe com os mesmos métodos serve (é a "tipagem estrutural", verificada pelo mypy).

`src/pulseops/clock.py, linhas 8–23`

```
 8 class Clock(Protocol):
 9     def now(self) -> datetime:
10         """Instante atual em UTC, com fuso (timezone-aware)."""
11         ...
12
13     def monotonic(self) -> float:
14         """Segundos de um relógio que nunca volta para trás; só serve para medir durações."""
15         ...
16
17
18 class SystemClock:
19     def now(self) -> datetime:
20         return datetime.now(UTC)
21
22     def monotonic(self) -> float:
23         return time.monotonic()
```

Protocolos usados no PulseOps:

Protocolo	Implementação real	Dublê nos testes
`Clock` (`clock.py`)	`SystemClock`	`FakeClock`: tempo parado que o teste avança
`Checker` (`checks/checker.py`)	`HttpChecker`	`HttpChecker` com `httpx.MockTransport` (`ScriptedTarget`)
`Notifier` (`notifications/webhook.py`)	`WebhookNotifier`	`RecordingNotifier`, que só anota o que receberia
`Resolver` (`security/transport.py`)	`SystemResolver`	`FakeResolver`, com respostas DNS fixas

`SystemClock` nem menciona `Clock`: ele simplesmente tem `now()` e `monotonic()`.

4.8 Exceções

Uma **exceção** interrompe o fluxo normal e sobe pela pilha de chamadas até alguém tratá-la com `try/except`.

- **Exceções próprias** (`services/errors.py`): `NotFoundError`, `ConflictError`, `ValidationFailedError`. Os serviços as levantam sem saber de HTTP; as rotas as traduzem em 404, 409 e 422.

- `raise ... from exc` preserva a causa original. Em `security/transport.py`, um `OSError` do DNS vira `httpcore.ConnectError` com a causa encadeada, e o verificador consegue olhar a cadeia para separar falha de DNS de falha TLS (`checker.py`, `_classify_connect_error`).
- **Barreira final**: em `Worker._run_one`, um `except Exception` em volta do verificador transforma qualquer erro inesperado em `INTERNAL_ERROR`, que não conta como falha do alvo. Sem isso, um bug do PulseOps derrubaria a tarefa e ninguém saberia.
- **Exceções de bibliotecas** que precisaram de cuidado viraram bugs reais: B4 (`UnicodeEncodeError` no Argon2) e B5 (`ZoneInfoNotFoundError` herda de `KeyError`, que o Pydantic não converte em erro de validação).

4.9 Context managers

Um *context manager* executa algo na entrada e garante algo na saída, mesmo se houver exceção, com `with` (ou `async with`).

Onde	O que garante
`async with sessions.begin() as session:`	`COMMIT` se o bloco terminar bem, `ROLLBACK` se houver exceção, e devolve a conexão
`async with asyncio.timeout(target.timeout_seconds):` (`checker.py`)	Cancela a requisição inteira se passar do prazo
`with log_context(monitor_id=..., run_key=...):` (`logs.py`)	Todo log dentro do bloco ganha esses campos; ao sair, o contexto anterior volta
`async with app.router.lifespan_context(app):` (testes)	Executa a inicialização e o encerramento da aplicação

`log_context` é um context manager **escrito no projeto**, com `@contextmanager`:

`src/pulseops/logs.py, linhas 35–43`

```
35 @contextmanager
36 def log_context(**fields: Any) -> Iterator[None]:
37     """Acrescenta campos a todos os logs emitidos dentro do bloco (inclusive em corrotinas)."""
38     current = _context.get() or {}
39     token = _context.set({**current, **fields})
40     try:
41         yield
42     finally:
43         _context.reset(token)
```

O `try/finally` é o que garante a restauração. `ContextVar` faz o contexto valer só para a tarefa `asyncio` atual: dois checks rodando ao mesmo tempo não misturam o `monitor_id` um do outro.

4.10 Tipagem

Anotações como `def apply(self, state: TrackerState, obs: Observation) -> Decision` não mudam a execução, mas permitem que o **mypy** confira o código antes de rodar (modo estrito, ADR 0009). O que o projeto usa:

- `X | None` para valores opcionais.

- `list[int]`, `dict[str, Any]`, `frozenset[tuple[str, int]]`.
- `Mapped[int]` nas entidades do SQLAlchemy 2.x, que liga o tipo Python à coluna.
- `Annotated[AsyncSession, Depends(get_db)]` em `web/deps.py`: o tipo e, junto, a instrução para o FastAPI de como obter o valor.
- `Protocol`, visto acima.
- `TypeVar` em `scripts/demo_flow.py` (`wait_for` devolve o mesmo tipo que a função testada devolve).

4.11 async/await

Um programa que faz rede passa a maior parte do tempo **esperando**. Com `asyncio`, uma única thread alterna entre várias tarefas: quando uma diz `await` (vou esperar a rede), outra pode andar.

- `async def` define uma **corrotina**; chamá-la não executa nada até alguém fazer `await` nela ou transformá-la em tarefa.
- `await algo()` pausa a corrotina atual até `algo` terminar.
- `asyncio.create_task(coro)` põe a corrotina para rodar "em paralelo" com as outras. O worker cria uma tarefa por check (`runner.py`, `_dispatch`).
- `asyncio.gather(*tarefas)` espera várias de uma vez (as entregas de um lote).
- `asyncio.Event` é um sinal: uma tarefa espera (`await evento.wait()`) até outra chamar `evento.set()`. O worker usa um para parar (`stop`) e outro para saber que uma vaga abriu (`_slot_freed`).

Regra de ouro: nada que bloqueie pode rodar dentro de uma corrotina. Um `time.sleep(5)` ou uma consulta síncrona ao banco travaria **todas** as tarefas. Por isso o projeto usa o driver assíncrono do PostgreSQL, o `httpx.AsyncClient` e `loop.getaddrinfo` (que roda a resolução de DNS em outra thread) em vez de `socket.getaddrinfo` direto.

4.12 Concorrência e seus limites

Concorrência não é ilimitada: cada check aberto usa uma conexão de rede, memória e, na gravação, uma conexão do banco. O worker nunca tem mais que `worker_concurrency` (padrão 20) checks em andamento: `_dispatch` reserva no banco só o número de vagas livres. O pool do banco tem 10 conexões (`db/session.py`), compartilhadas por todas as tarefas: se mais tarefas quiserem gravar ao mesmo tempo, esperam na fila do pool.

Outro limite: **uma `AsyncSession` nunca é compartilhada entre tarefas**. A sessão guarda o estado de uma transação; duas tarefas usando a mesma sessão misturariam comandos de transações diferentes. Cada função de `services/processing.py` abre a própria sessão com `async with sessions.begin()`.

O que asyncio **não** dá: paralelismo de CPU. Tudo roda numa thread; se o trabalho fosse cálculo pesado, seria preciso processos. Para o PulseOps, que espera rede, asyncio é suficiente (*só para comparação*: threads e `multiprocessing` não são usados).

4.13 Injeção de dependências

Injetar uma dependência é **receber de fora** aquilo de que uma peça precisa, em vez de criá-lo dentro dela. Dois estilos aparecem:

1. **Pelo construtor ou por parâmetro.** `Worker(sessions=..., checker=..., notifier=..., clock=..., settings=...)`; `create_app(settings, sessions=..., clock=..., resolver=...)`. Em produção, `worker/__main__.py` passa os objetos reais; nos testes, `tests/support_db.make_worker` passa dublês.
2. **Pelo FastAPI (`Depends`).** Em `web/deps.py`, `Db = Annotated[AsyncSession, Depends(get_db)]` diz: "para obter este parâmetro, chame `get_db`". Uma rota que declara `db: Db` recebe uma sessão nova por requisição, fechada no fim. `ApiAuth` encadeia: para autenticar, precisa da sessão do banco e do estado da aplicação, e o FastAPI resolve a cadeia sozinho.

O ganho prático: os testes controlam o tempo (`FakeClock`), a rede (`ScriptedTarget`, `FakeResolver`) e os alertas (`RecordingNotifier`) sem mudar uma linha do código de produção.

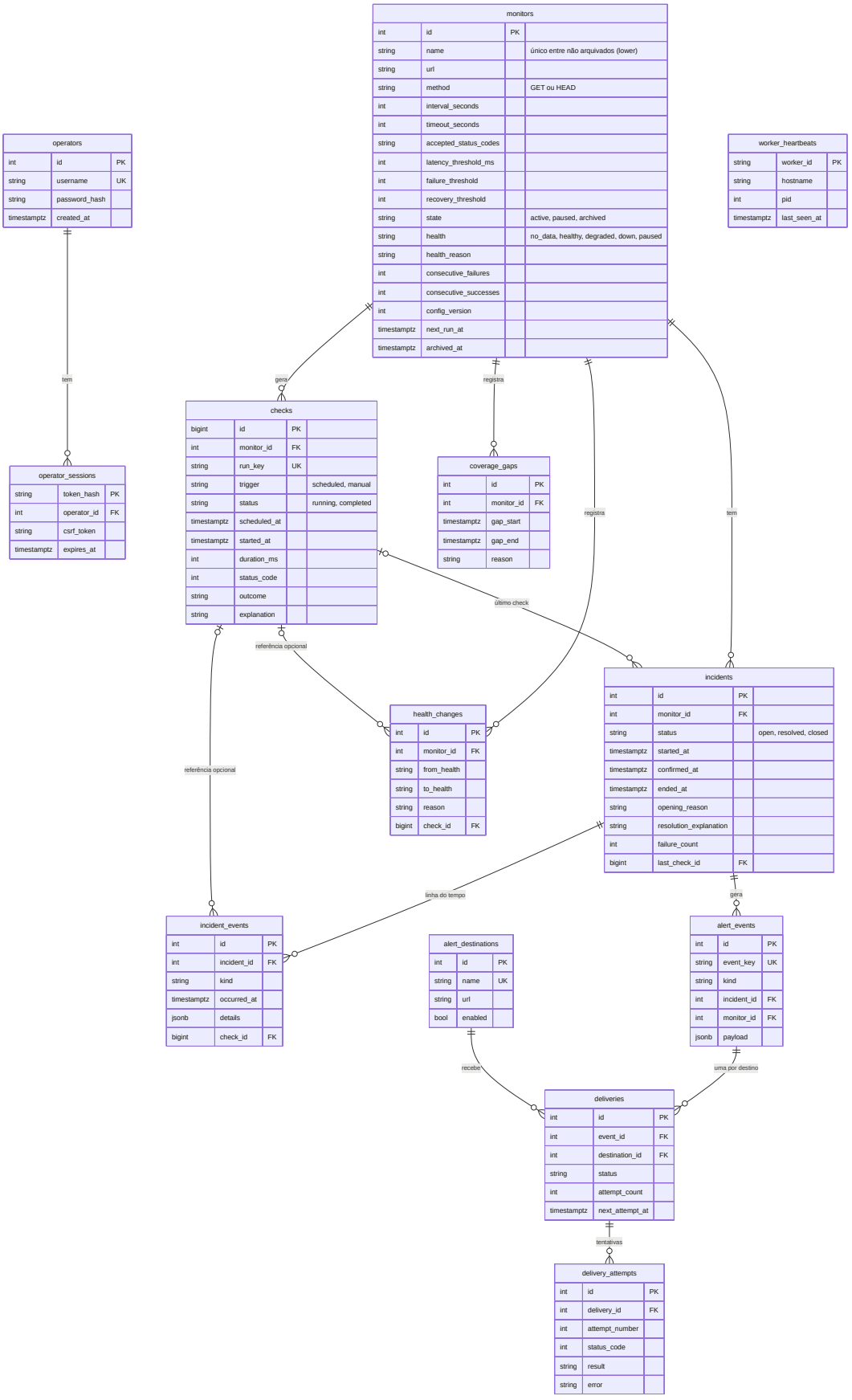
5 Parte F — Banco de dados e SQL

O PostgreSQL é a peça que dá confiança ao PulseOps. Ele guarda o histórico, e também **garante regras**: algumas promessas do sistema (“um incidente aberto por monitor”, “um check em andamento por monitor”) não dependem só do código Python estar certo.

5.1 Modelo relacional

Um banco **relacional** organiza dados em **tabelas**. Cada **linha** é um registro e cada **coluna** tem um tipo (`integer`, `varchar`, `timestamptz`...). Tabelas se ligam por valores: a coluna `checks.monitor_id` guarda o `id` do monitor ao qual o check pertence.

O diagrama abaixo mostra as 13 tabelas e como se relacionam (é o mesmo diagrama de `docs/arquitetura.md`, com os atributos expandidos lá).



Leitura do diagrama, pelos grupos:

Grupo	Tabelas	Papel
Acesso	`operators`, `operator_sessions`	quem pode entrar e as sessões abertas

Grupo	Tabelas	Papel
Configuração	`monitors`, `alert_destinations`	o que verificar e para onde avisar
Fatos observados	`checks`, `coverage_gaps`	cada verificação feita e os períodos sem verificação
Decisões	`health_changes`, `incidents`, `incident_events`	o que o sistema concluiu a partir dos fatos, com o motivo
Alertas	`alert_events`, `deliveries`, `delivery_attempts`	outbox: o evento, a entrega por destino e cada tentativa
Processo	`worker_heartbeats`	último sinal de vida do worker

A separação entre **fatos** e **decisões** é proposital. Um check é algo que aconteceu e não muda depois de concluído. Uma mudança de saúde é uma conclusão tirada de vários checks, e guarda o motivo em texto. Assim, dá para auditar qualquer decisão voltando aos fatos que a geraram.

5.2 Chaves primárias e estrangeiras

- **Chave primária** (*primary key*, PK): identifica cada linha de forma única. Em quase todas as tabelas é uma coluna `id` numerada pelo próprio banco. Duas exceções: `operator_sessions` usa o `token_hash` (o hash do cookie, que já é único) e `worker_heartbeats` usa o `worker_id`.
- **Chave estrangeira** (*foreign key*, FK): diz que o valor de uma coluna precisa existir em outra tabela. `checks.monitor_id` só aceita `id` de monitores que existem. O banco recusa um check "órfão".

O `ondelete` da FK diz o que acontece quando a linha apontada é apagada:

`src/pulseops/db/models.py, linhas 160–163`

```
160     id: Mapped[int] = mapped_column(BigInteger, primary_key=True)
161     monitor_id: Mapped[int] = mapped_column(ForeignKey("monitors.id", ondelete="RESTRICT"))
162     # Identidade da execução agendada: "{monitor_id}:{scheduled_at ISO UTC}".
163     run_key: Mapped[str] = mapped_column(String(80), unique=True)
```

Regra	Onde	Por quê
`RESTRICT`	`checks.monitor_id`, `incidents.monitor_id`, entregas…	monitores nunca são apagados, só arquivados; se alguém tentar apagar um monitor com histórico, o banco recusa
`CASCADE`	`operator_sessions.operator_id`	sessões não têm sentido sem a operadora
`SET NULL`	`incidents.last_check_id`, `incident_events.check_id`, `health_changes.check_id`	a retenção apaga checks antigos, mas incidentes ficam para sempre; a referência vira nula em vez de impedir a limpeza

5.3 Constraints: o banco como segunda linha de defesa

Uma **constraint** (restrição) é uma regra que o banco verifica em toda gravação. Se a regra é violada, o comando inteiro falha. O PulseOps usa três tipos.

CHECK — condição sobre os valores da própria linha:

src/pulseops/db/models.py, linhas 67–86

```
67    __table_args__ = (
68        CheckConstraint("method IN ('GET', 'HEAD')", name="method"),
69        CheckConstraint("state IN ('active', 'paused', 'archived')", name="state"),
70        CheckConstraint(
71            "health IN ('no_data', 'healthy', 'degraded', 'down', 'paused')", name="health"
72        ),
73        # Limites absolutos. O mínimo de 30 s do modo normal é validado na aplicação,
74        # porque o modo demonstração permite 2 s.
75        CheckConstraint("interval_seconds BETWEEN 2 AND 86400", name="interval"),
76        CheckConstraint("timeout_seconds BETWEEN 1 AND 30", name="timeout"),
77        CheckConstraint("timeout_seconds < interval_seconds", name="timeout_lt_interval"),
78        CheckConstraint(
79            "latency_threshold_ms BETWEEN 50 AND timeout_seconds * 1000", name="latency"
80        ),
81        CheckConstraint("failure_threshold BETWEEN 1 AND 10", name="failure_threshold"),
82        CheckConstraint("recovery_threshold BETWEEN 1 AND 10", name="recovery_threshold"),
83        CheckConstraint(
84            "consecutive_failures >= 0 AND consecutive_successes >= 0", name="counters"
85        ),
86        CheckConstraint("(state = 'archived') = (archived_at IS NOT NULL)", name="archived_at"),
```

Linha a linha do que importa aqui:

- 68–72: listas fechadas de valores. `state` só pode ser `active`, `paused` ou `archived`. Um erro de digitação no código (`"actve"`) é recusado pelo banco.
- 75: o intervalo fica entre 2 s e 1 dia. O mínimo de 30 s do modo normal é conferido na aplicação (o comentário das linhas 73–74 explica: o modo demonstração aceita 2 s).
- 77: o timeout precisa ser menor que o intervalo, senão um check ainda estaria rodando quando o próximo começasse.
- 78–80: o limite de lentidão fica entre 50 ms e o timeout. Um limite maior que o timeout nunca seria alcançado.
- 86: `(state = 'archived') = (archived_at IS NOT NULL)` compara dois booleanos: a data de arquivamento existe **se e somente se** o monitor está arquivado.

UNIQUE — nenhum par de linhas pode repetir o valor. Exemplos: `checks.run_key`, `alert_events.event_key` e o par (`event_id`, `destination_id`) em `deliveries`.

Índice único parcial — único só entre as linhas que satisfazem uma condição. É a ferramenta mais importante do esquema:

src/pulseops/db/models.py, linhas 147–153

```
147        # No máximo um check em andamento por monitor: o banco impede sobreposição (CB3).
148        Index(
149            "uq_checks_one_running_per_monitor",
150            "monitor_id",
151            unique=True,
152            postgresql_where=text("status = 'running'"),
153        ),
```

`WHERE status = 'running'` faz o índice conter só os checks em andamento. Como ele é `unique` por `monitor_id`, **no máximo um check em andamento por monitor**. Checks concluídos ficam fora do índice, então um monitor pode ter milhares deles.

O mesmo padrão aparece em outros dois lugares:

Índice	Garante	Requisito
`uq_checks_one_running_per_monitor`	um check em andamento por monitor	CB3
`uq_incidents_one_open_per_monitor`	um incidente aberto por monitor	RF03.11
`uq_monitors_active_name`	nome único (sem diferenciar maiúsculas) entre monitores não arquivados	RF01.7

O último usa `lower(name)` dentro do índice. "API Pagamentos" e "api pagamentos" viram o mesmo valor e colidem. Um monitor arquivado sai do índice, então o nome dele pode ser reutilizado.

Por que repetir no banco o que o código já faz? O código confere antes de gravar, mas entre a conferência e a gravação outra transação pode ter gravado a mesma coisa (a "corrida" da seção de concorrência). Só o banco vê todas as transações. Os testes `test_database_rejects_second_running_check` e `test_database_rejects_second_open_incident` (em `tests/integration/test_consistency.py`) tentam inserir a segunda linha diretamente, sem passar pelo código, e conferem que o PostgreSQL recusa.

5.4 Índices

Um **índice** é uma estrutura auxiliar (no PostgreSQL, por padrão, uma árvore B) que permite achar linhas sem ler a tabela inteira, como o índice remissivo de um livro. Custa espaço e deixa as gravações um pouco mais lentas, então cada índice precisa de uma consulta que o justifique.

Índice	Colunas	Consulta que atende
`ix_monitors_due` (parcial, `state = 'active'`)	`next_run_at`	o agendador: "monitores ativos com execução vencida"
`ix_checks_monitor_history`	`monitor_id, started_at, id`	histórico de um monitor, paginado
`ix_checks_started_at`	`started_at`	retenção: checks antigos de todos os monitores
`ix_incidents_confirmed`	`confirmed_at, id`	lista geral de incidentes
`ix_deliveries_due` (parcial, pendentes)	`next_attempt_at`	o entregador: "entregas vencidas"
`ix_incidents_last_check` e similares	colunas com `SET NULL`	ao apagar um check, o banco precisa achar quem aponta para ele

A ordem das colunas num índice composto importa. `(monitor_id, started_at, id)` serve para "checks **deste** monitor, ordenados por data", porque primeiro filtra pelo monitor e depois já encontra as linhas na ordem certa. Não serviria para "checks de todos os monitores de ontem"; para isso existe

`ix_checks_started_at`.

O último grupo é um detalhe fácil de esquecer: o PostgreSQL **não** cria índice sozinho nas colunas de chave estrangeira. Sem `ix_incidents_last_check`, apagar 5000 checks na retenção faria o banco varrer a tabela de incidentes 5000 vezes para aplicar o `SET NULL`.

5.5 ORM: tabelas como classes Python

Um **ORM** (*Object-Relational Mapper*) liga classes Python a tabelas. O PulseOps usa o SQLAlchemy 2.x no estilo com anotações de tipo:

src/pulseops/db/models.py, linhas 132–135

```python
class Check(Base):
    __tablename__ = "checks"
    __table_args__ = (
        CheckConstraint("status IN ('running', 'completed')", name="status"),
```

src/pulseops/db/models.py, linhas 160–173

```python
    id: Mapped[int] = mapped_column(BigInteger, primary_key=True)
    monitor_id: Mapped[int] = mapped_column(ForeignKey("monitors.id", ondelete="RESTRICT"))
    # Identidade da execução agendada: "{monitor_id}:{scheduled_at ISO UTC}".
    run_key: Mapped[str] = mapped_column(String(80), unique=True)
    trigger: Mapped[str] = mapped_column(String(10), default="scheduled")
    status: Mapped[str] = mapped_column(String(10), default="running")
    config_version: Mapped[int] = mapped_column(Integer)
    scheduled_at: Mapped[datetime] = mapped_column(Timestamp)
    started_at: Mapped[datetime] = mapped_column(Timestamp)
    finished_at: Mapped[datetime | None] = mapped_column(Timestamp)
    duration_ms: Mapped[int | None] = mapped_column(Integer)
    status_code: Mapped[int | None] = mapped_column(Integer)
    outcome: Mapped[str | None] = mapped_column(String(20))
    explanation: Mapped[str | None] = mapped_column(String(300))
```

- `class Check(Base)`: cada instância é uma linha da tabela `checks`.
- `Mapped[int]`: o tipo que o Python (e o mypy) enxerga.
- `mapped_column(...)`: detalhes da coluna no banco. `Mapped[int | None]` vira uma coluna que aceita `NULL`; `Mapped[int]` vira `NOT NULL`.
- `BigInteger` nos ids de tabelas que crescem sem parar (checks, eventos): um `integer` comum acaba em cerca de 2,1 bilhões.

A conexão e a fábrica de sessões ficam em `src/pulseops/db/session.py`:

src/pulseops/db/session.py, linhas 13–24

```python
def create_engine(database_url: str, *, pool_size: int = 10) -> AsyncEngine:
    return create_async_engine(
        database_url,
        pool_size=pool_size,
        pool_pre_ping=True,  # descarta conexões mortas (por exemplo, após o banco reiniciar)
        connect_args={"connect_timeout": 5},
    )


def create_session_factory(engine: AsyncEngine) -> SessionFactory:
    # expire_on_commit=False: objetos continuam legíveis depois do commit, sem nova consulta.
    return async_sessionmaker(engine, expire_on_commit=False)
```

- `pool_size`: quantas conexões o processo mantém abertas para reaproveitar.

- `pool_pre_ping=True`: antes de usar uma conexão do pool, confere se ela ainda está viva. Sem isso, depois de o banco reiniciar, a primeira consulta falharia.
- `expire_on_commit=False`: depois do `commit`, os objetos continuam legíveis sem ir ao banco de novo. O worker lê `monitor.id` depois de fechar a transação, então isso evita consultas extras (e erros em código assíncrono).

O que o ORM não faz por você: ele não decide quando a transação começa e termina, nem quais linhas travar. Essas decisões estão escritas explicitamente no código (próximas seções).

5.6 O SQL que realmente roda

Os trechos de SQL a seguir **não foram escritos à mão**. O script `study/captura_sql.py` é um plugin do pytest que escuta todos os comandos enviados ao PostgreSQL durante os testes de integração e salva os de interesse. A única mudança feita no texto é de espaçamento: quebras de linha antes de palavras como `FROM` e `WHERE`.

Os marcadores `%(nome)s` são **parâmetros**: o valor vai separado do comando, e o banco nunca o interpreta como SQL. É isso que torna o PulseOps imune a injeção de SQL nesses pontos, e é testado em `test_sql_injection_attempts_are_plain_data` (em `tests/integration/test_dashboard.py`). O `::VARCHAR` depois de cada parâmetro é um *cast* (conversão de tipo) que o SQLAlchemy acrescenta.

5.6.1 Reservar monitores vencidos

Código Python (`src/pulseops/services/processing.py`):

`src/pulseops/services/processing.py, linhas 60–70`

```python
    running = exists().where(Check.monitor_id == Monitor.id, Check.status == "running")
    async with sessions.begin() as session:
        monitors = (
            await session.scalars(
                select(Monitor)
                .where(Monitor.state == "active", Monitor.next_run_at <= now, ~running)
                .order_by(Monitor.next_run_at, Monitor.id)
                .limit(limit)
                .with_for_update(skip_locked=True)
            )
        ).all()
```

SQL gerado:

`SQL enviado ao PostgreSQL: reservar-monitores`

```sql
SELECT monitors.id, monitors.name, monitors.description, monitors.url, monitors.method,
    monitors.interval_seconds, monitors.timeout_seconds, monitors.accepted_status_codes,
    monitors.latency_threshold_ms, monitors.failure_threshold, monitors.recovery_threshold, monitors.state,
    monitors.health, monitors.health_reason, monitors.health_changed_at, monitors.consecutive_failures,
    monitors.consecutive_successes, monitors.first_failure_at, monitors.config_version, monitors.next_run_at,
    monitors.check_requested, monitors.last_check_at, monitors.created_at, monitors.updated_at,
    monitors.archived_at
  FROM monitors
  WHERE monitors.state = %(state_1)s::VARCHAR
  AND monitors.next_run_at <= %(next_run_at_1)s::TIMESTAMP WITH TIME ZONE
  AND NOT (EXISTS (SELECT *
  FROM checks
  WHERE checks.monitor_id = monitors.id
  AND checks.status = %(status_1)s::VARCHAR))
  ORDER BY monitors.next_run_at, monitors.id
```

```
  LIMIT %(param_1)s::INTEGER
  FOR UPDATE SKIP LOCKED
```

Parte por parte:

- `WHERE monitors.state = ... AND monitors.next_run_at <= ...`: ativos e com a próxima execução no passado (ou agora).
- `NOT (EXISTS (SELECT * FROM checks WHERE ... status = 'running'))`: pula monitores que já têm um check em andamento. `EXISTS` responde "há pelo menos uma linha?" e para na primeira que encontrar.
- `ORDER BY monitors.next_run_at, monitors.id`: os mais atrasados primeiro; o `id` desempata para a ordem ser sempre a mesma.
- `LIMIT`: no máximo o número de vagas livres do worker.
- `FOR UPDATE SKIP LOCKED`: trava as linhas escolhidas até o fim da transação e **pula** as que outra transação já travou. Explicação completa na seção de concorrência.

5.6.2 Gravar o check como "em andamento", sem duplicar

`src/pulseops/services/processing.py, linhas 105–123`

```
105     run_key = make_run_key(monitor.id, scheduled_at)
106     check_id = await session.scalar(
107         insert(Check)
108         .values(
109             monitor_id=monitor.id,
110             run_key=run_key,
111             trigger=trigger,
112             status="running",
113             config_version=monitor.config_version,
114             scheduled_at=scheduled_at,
115             started_at=now,
116         )
117         .on_conflict_do_nothing(index_elements=["run_key"])
118         .returning(Check.id)
119     )
120     monitor.next_run_at = max(scheduled_at + interval, now + timedelta(seconds=1))
121     if check_id is None:
122         # Esta execução já existe (reprocessamento da mesma identidade): não duplica.
123         return None
```

`SQL enviado ao PostgreSQL: inserir-check`

```
INSERT INTO checks (monitor_id, run_key, trigger, status, config_version, scheduled_at, started_at)
  VALUES (%(monitor_id)s::INTEGER, %(run_key)s::VARCHAR, %(trigger)s::VARCHAR, %(status)s::VARCHAR, %
   (config_version)s::INTEGER, %(scheduled_at)s::TIMESTAMP WITH TIME ZONE, %(started_at)s::TIMESTAMP WITH
   TIME ZONE)
  ON CONFLICT (run_key) DO NOTHING
  RETURNING checks.id
```

- `ON CONFLICT (run_key) DO NOTHING`: se já existe um check com essa identidade, o banco não insere e não dá erro.
- `RETURNING checks.id`: devolve o id criado. Se nada foi inserido, não volta nenhuma linha, o Python recebe `None` (linha 121) e entende que essa execução já existia.

O `run_key` é `"{monitor_id}:{horário agendado}"` (linha 46–47 do mesmo arquivo). A mesma execução agendada tem sempre o mesmo `run_key`, então reprocessá-la não cria um segundo check. Teste: `test_same_run_identity_cannot_be_reserved_twice`.

5.6.3 Histórico paginado por cursor

SQL enviado ao PostgreSQL: historico-cursor

```
SELECT checks.id, checks.monitor_id, checks.run_key, checks.trigger, checks.status, checks.config_version,
    checks.scheduled_at, checks.started_at, checks.finished_at, checks.duration_ms, checks.status_code,
    checks.outcome, checks.explanation
  FROM checks
  WHERE checks.monitor_id = %(monitor_id_1)s::INTEGER
  AND checks.status = %(status_1)s::VARCHAR
  AND (checks.started_at, checks.id) < (%(param_1)s::TIMESTAMP WITH TIME ZONE, %(param_2)s::INTEGER)
  ORDER BY checks.started_at DESC, checks.id DESC
  LIMIT %(param_3)s::INTEGER
```

`(checks.started_at, checks.id) < (..., ...)` compara **pares**: primeiro pela data; se empatar, pelo id. É a "paginação por cursor" (*keyset pagination*): o cursor que a API devolve é o par do último item da página, codificado em base64 (`encode_cursor` em `src/pulseops/services/queries.py`). A próxima página começa logo depois dele.

Por que não `OFFSET`? Com `OFFSET 50`, o banco precisa ler e descartar 50 linhas, e se um check novo chegar entre uma página e outra, todo mundo "anda" uma posição: um item aparece duas vezes. Com o cursor, a posição é definida pelo próprio dado. O `LIMIT` pede um item a mais que o tamanho da página só para saber se há próxima página. Teste: `test_cursor_pagination_is_stable_and_complete`.

5.6.4 Métricas

SQL enviado ao PostgreSQL: metricas-contagem

```
SELECT checks.outcome, count(*) AS count_1
  FROM checks
  WHERE checks.monitor_id = %(monitor_id_1)s::INTEGER
  AND checks.status = %(status_1)s::VARCHAR
  AND checks.started_at >= %(started_at_1)s::TIMESTAMP WITH TIME ZONE
  AND checks.started_at < %(started_at_2)s::TIMESTAMP WITH TIME ZONE
  GROUP BY checks.outcome
```

`GROUP BY checks.outcome` junta as linhas por resultado e `count(*)` conta cada grupo. O resultado tem uma linha por tipo de resultado (`healthy`, `timeout`...). A janela é `started_at >= início AND started_at < fim`: fechada no começo e aberta no fim, para que janelas vizinhas não contem o mesmo check duas vezes.

SQL enviado ao PostgreSQL: metricas-latencia

```
SELECT count(checks.duration_ms) AS count_1, avg(checks.duration_ms) AS avg_1, percentile_disc(%
    (percentile_disc_2)s) WITHIN GROUP (ORDER BY checks.duration_ms) AS percentile_disc_1
  FROM checks
  WHERE checks.monitor_id = %(monitor_id_1)s::INTEGER
  AND checks.status = %(status_1)s::VARCHAR
  AND checks.started_at >= %(started_at_1)s::TIMESTAMP WITH TIME ZONE
  AND checks.started_at < %(started_at_2)s::TIMESTAMP WITH TIME ZONE
  AND checks.outcome IN (%(outcome_1_1)s::VARCHAR, %(outcome_1_2)s::VARCHAR, %(outcome_1_3)s::VARCHAR)
```

`percentile_disc(0.95) WITHIN GROUP (ORDER BY duration_ms)` ordena as durações e devolve o primeiro valor cuja posição acumulada chega a 95%. É o método *nearest-rank* (ADR 0011). O filtro `outcome IN (...)` deixa de fora timeouts e erros de conexão, que não têm duração de resposta. O teste `test_sql_percentile_matches_nearest_rank` compara o resultado do SQL com a função Python `nearest_rank` de `src/pulseops/domain/metrics.py`.

5.6.5 Retenção

src/pulseops/services/processing.py, linhas 267–279

```python
async def purge_old_checks(
    sessions: SessionFactory, *, now: datetime, retention_days: int, batch_size: int = 5000
) -> int:
    """Retenção (CB6): apaga checks concluídos antigos em lotes. Incidentes ficam."""
    cutoff = now - timedelta(days=retention_days)
    async with sessions.begin() as session:
        ids = (
            select(Check.id)
            .where(Check.started_at < cutoff, Check.status == "completed")
            .limit(batch_size)
        )
        result = await session.execute(delete(Check).where(Check.id.in_(ids.scalar_subquery())))
        return int(getattr(result, "rowcount", 0) or 0)
```

SQL enviado ao PostgreSQL: retencao

```sql
DELETE
  FROM checks
  WHERE checks.id IN (SELECT checks.id
  FROM checks
  WHERE checks.started_at < %(started_at_1)s::TIMESTAMP WITH TIME ZONE
  AND checks.status = %(status_1)s::VARCHAR
  LIMIT %(param_1)s::INTEGER)
  RETURNING checks.id
```

Apaga em **lotes** de até 5000 checks concluídos mais antigos que o prazo. Um `DELETE` de milhões de linhas numa transação só seguraria travas por muito tempo. O `RETURNING checks.id` não está no código Python: foi o SQLAlchemy que o acrescentou, porque um `delete()` do ORM sincroniza os objetos já carregados na sessão. Incidentes não são apagados, e as referências a checks apagados viram `NULL` (o `SET NULL` visto antes). Teste: `test_old_checks_are_purged_and_incidents_kept`.

5.7 Transações, commit e rollback

Uma **transação** é um grupo de comandos que o banco trata como uma unidade: ou **todos** valem (`COMMIT`), ou **nenhum** vale (`ROLLBACK`). Isso é o "A" de ACID (*atomicidade*). As outras letras: **C**onsistência (constraints valem no fim de toda transação), **I**solamento (transações simultâneas não veem o trabalho pela metade umas das outras) e **D**urabilidade (o que foi confirmado sobrevive a uma queda).

No PulseOps, o padrão é `async with sessions.begin() as session:`.

src/pulseops/services/processing.py, linhas 139–153

```python
async def record_result(
    sessions: SessionFactory, run: ClaimedRun, observation: CheckObservation
) -> bool:
    """Grava o resultado e aplica as regras de domínio em uma única transação (CB1).

    Devolve False se esta execução já tinha sido gravada (reprocessamento ignorado).
    """
    with log_context(monitor_id=run.monitor_id, run_key=run.run_key):
        async with sessions.begin() as session:
            # Trava o monitor: serializa com edição/pausa feitas ao mesmo tempo pela API.
            monitor = await session.get(Monitor, run.monitor_id, with_for_update=True)
            check = await session.get(Check, run.check_id, with_for_update=True)
            if monitor is None or check is None or check.status != "running":
```

```
152                 logger.info("resultado ignorado: execução já gravada")
153                 return False
```

- Entrar no bloco abre a transação.
- Sair do bloco normalmente faz `COMMIT`.
- Sair por exceção faz `ROLLBACK` e deixa a exceção seguir.

`record_result` é o melhor exemplo de por que isso importa. Dentro de uma única transação ele: marca o check como concluído, atualiza os contadores do monitor, talvez abre ou resolve um incidente, grava a mudança de saúde **e** cria o evento de alerta com as entregas. Se o banco cair no meio, nada disso fica pela metade: não existe "incidente aberto sem alerta" nem "alerta sem incidente". Testes: `test_outbox_event_is_atomic_with_incident` (em `tests/integration/test_delivery.py`) e `test_database_failure_while_recording_does_not_count_as_target_failure`.

Transações curtas (ADR 0003). O ciclo de um check usa **duas** transações, com a requisição HTTP entre elas e fora de ambas:

1. `claim_due_runs`: reserva e grava o check `running`. Commit.
2. Requisição HTTP ao alvo (até `timeout_seconds`), sem transação aberta.
3. `record_result`: grava o resultado e aplica as regras. Commit.

Se a requisição ficasse dentro da transação, cada check lento seguraria uma conexão e travas por até 30 segundos. Com 100 monitores lentos, o pool de conexões acabaria. O preço da separação é tratar o caso em que o worker cai entre os passos 1 e 3: o check fica `running` para sempre. `mark_abandoned_runs` (linhas 239–264 de `processing.py`) encontra esses checks e os marca como `internal_error`, que **não** conta como falha do alvo.

Na API, a sessão é aberta por requisição (`get_db` em `src/pulseops/web/deps.py`) e as rotas que alteram dados chamam `commit()` explicitamente. O que não foi confirmado é desfeito quando a sessão fecha.

5.8 Migrações com Alembic

O esquema do banco evolui junto com o código. Uma **migração** é um arquivo Python com duas funções: `upgrade()` (aplica a mudança) e `downgrade()` (desfaz). O Alembic anota na tabela `alembic_version` qual foi a última aplicada.

O PulseOps tem uma migração só, `migrations/versions/0001_esquema_inicial.py`. Ela foi **gerada automaticamente** pelo Alembic (`alembic revision --autogenerate`), comparando os modelos com um banco vazio, e depois revisada. Por isso o volume 2 a explica por padrão, e não linha a linha.

Dois testes cuidam dela (`tests/integration/test_migrations.py`):

- `test_upgrade_on_empty_database_creates_all_tables`: aplica a migração num banco vazio e usa `compare_metadata` para conferir que o resultado é **idêntico** aos modelos. Se alguém mudar um modelo e esquecer a migração, o teste falha.
- `test_downgrade_and_upgrade_again`: desfaz tudo (sobra só `alembic_version`) e aplica de novo.

O CI repete o ciclo `upgrade` → `downgrade` → `upgrade` contra um PostgreSQL de verdade.

Limite honesto: com uma migração só, o projeto nunca passou por uma **alteração** de esquema com dados existentes (renomear coluna, preencher coluna nova). Esse é o caso difícil de migrações, e não está exercitado aqui.

5.9 Concorrência

Concorrência é quando duas coisas acontecem "ao mesmo tempo" e disputam os mesmos dados. No PulseOps, há três fontes:

- o worker roda até `PULSEOPS_WORKER_CONCURRENCY` checks em paralelo (20 por padrão);
- a API e o worker alteram o mesmo monitor (a operadora pausa enquanto um check roda);
- alguém pode, por engano, iniciar um segundo worker.

5.9.1 Travas de linha: `FOR UPDATE`

`SELECT ... FOR UPDATE` trava as linhas lidas até o fim da transação. Outra transação que tente travar a mesma linha **espera**.

SQL enviado ao PostgreSQL: travar-monitor

```
SELECT monitors.id, monitors.name, monitors.description, monitors.url, monitors.method,
    monitors.interval_seconds, monitors.timeout_seconds, monitors.accepted_status_codes,
    monitors.latency_threshold_ms, monitors.failure_threshold, monitors.recovery_threshold, monitors.state,
    monitors.health, monitors.health_reason, monitors.health_changed_at, monitors.consecutive_failures,
    monitors.consecutive_successes, monitors.first_failure_at, monitors.config_version, monitors.next_run_at,
    monitors.check_requested, monitors.last_check_at, monitors.created_at, monitors.updated_at,
    monitors.archived_at
  FROM monitors
  WHERE monitors.id = %(pk_1)s::INTEGER FOR UPDATE
```

Esse comando sai de `session.get(Monitor, run.monitor_id, with_for_update=True)` na linha 149 de `processing.py`. Ele serializa `record_result` com a pausa feita pela API, que também trava o monitor (`get_monitor(..., lock=True)` em `services/monitors.py`). Uma das duas termina primeiro; a outra vê o resultado.

5.9.2 Pular o que está ocupado: `SKIP LOCKED`

Com `SKIP LOCKED`, em vez de esperar, a consulta **pula** linhas travadas. É o padrão de "fila no banco": dois consumidores pegam itens diferentes, sem brigar. O teste `test_concurrent_claims_never_take_the_same_monitor` dispara reservas em paralelo e confere que nenhum monitor é reservado duas vezes.

5.9.3 A versão da configuração

Travar não resolve tudo. Considere: o check começa às 10:00:00 com timeout de 5 s; às 10:00:02 a operadora muda o limite de falhas; às 10:00:05 o check termina. O resultado foi medido com a configuração **antiga**. Deve contar?

O PulseOps responde "não": cada check guarda o `config_version` do momento da reserva, e `record_result` só aplica as regras se ele ainda for igual ao do monitor (linhas 164–167 de `processing.py`). Edição, pausa, retomada e arquivamento incrementam a versão. O bug B6 (registro de bugs) era justamente a pausa e a retomada **não** incrementarem: um check iniciado antes da pausa

contava depois da retomada. Testes: `test_result_of_run_started_before_config_change_does_not_count` e `test_result_of_run_started_before_pause_and_resume_does_not_count` (em `tests/integration/test_incident_lifecycle.py`).

5.9.4 Um worker só: advisory lock

src/pulseops/worker/**main**.py, linhas 24–32

```python
# Chave do advisory lock que garante um único worker por banco (ADR 0002).
WORKER_LOCK_KEY = 0x50554C53  # "PULS"


async def try_acquire_worker_lock(conn: AsyncConnection) -> bool:
    """Lock de sessão do PostgreSQL: só uma conexão por banco consegue segurá-lo."""
    got = await conn.scalar(text("SELECT pg_try_advisory_lock(:k)"), {"k": WORKER_LOCK_KEY})
    await conn.commit()  # o lock é de sessão: continua valendo depois do commit
    return bool(got)
```

Um **advisory lock** é uma trava sem tabela: um número que só uma conexão por vez pode segurar. O worker tenta pegá-lo ao iniciar com `pg_try_advisory_lock`, que responde na hora (`true` ou `false`) em vez de esperar. Se outro worker já o tem, este registra o erro e sai com código 1. O lock é **de sessão**: dura enquanto a conexão estiver aberta, e o PostgreSQL o libera sozinho se o processo morrer. Teste: `test_second_worker_is_refused`.

5.9.5 Entregas: reserva com prazo

SQL enviado ao PostgreSQL: reservar-entregas

```sql
SELECT deliveries.id, deliveries.event_id, deliveries.destination_id, deliveries.status,
    deliveries.attempt_count, deliveries.next_attempt_at, deliveries.last_error, deliveries.delivered_at,
    deliveries.created_at, deliveries.updated_at, alert_events.id AS id_1, alert_events.event_key,
    alert_events.kind, alert_events.incident_id, alert_events.monitor_id, alert_events.payload,
    alert_events.created_at AS created_at_1, alert_destinations.id AS id_2, alert_destinations.name,
    alert_destinations.url, alert_destinations.enabled, alert_destinations.created_at AS created_at_2,
    alert_destinations.updated_at AS updated_at_1
  FROM deliveries
  JOIN alert_events ON alert_events.id = deliveries.event_id
  JOIN alert_destinations ON alert_destinations.id = deliveries.destination_id
  WHERE deliveries.status IN (%(status_1_1)s::VARCHAR, %(status_1_2)s::VARCHAR)
  AND deliveries.next_attempt_at <= %(next_attempt_at_1)s::TIMESTAMP WITH TIME ZONE
  ORDER BY deliveries.next_attempt_at, deliveries.id
  LIMIT %(param_1)s::INTEGER
  FOR UPDATE OF deliveries SKIP LOCKED
```

`FOR UPDATE OF deliveries SKIP LOCKED` trava só as linhas de `deliveries`, não as do evento e do destino que vieram no `JOIN`. A reserva muda o status para `sending` e põe em `next_attempt_at` o fim do prazo de reserva (*lease*). Se o worker morrer durante o envio, a reserva vence e outra rodada pega a entrega de novo. Isso pode gerar **entrega duplicada** (o destino recebeu, mas o worker morreu antes de gravar), e por isso cada evento leva um `event_id` estável para o destino descartar repetições (RF06.8).

SQL enviado ao PostgreSQL: gravar-entrega

```sql
UPDATE deliveries SET status=%(status)s::VARCHAR, last_error=%(last_error)s::VARCHAR, delivered_at=%
    (delivered_at)s::TIMESTAMP WITH TIME ZONE, updated_at=%(updated_at)s::TIMESTAMP WITH TIME ZONE
  WHERE deliveries.id = %(id_1)s::BIGINT
  AND deliveries.status = %(status_1)s::VARCHAR
  AND deliveries.attempt_count = %(attempt_count_1)s::INTEGER
```

A gravação do resultado só altera a linha se ela ainda estiver `sending` **com o mesmo número de tentativa**. Se a reserva venceu e outra tentativa já começou, esta gravação atrasada não sobrescreve o estado mais novo. É uma forma de **controle otimista**: em vez de travar antes, confere na hora de gravar.

5.10 Resumo da parte F

Garantia	Mecanismo	Onde
Um check em andamento por monitor	índice único parcial + `NOT EXISTS` na reserva	`models.py`, `processing.py`
Um incidente aberto por monitor	índice único parcial	`models.py`
Execução não duplicada	`run_key` único + `ON CONFLICT DO NOTHING`	`processing.py`
Incidente e alerta juntos ou nada	uma transação em `record_result`	`processing.py`, `alerts.py`
Resultado velho não decide	`config_version`	`processing.py`, `monitors.py`
Um worker por banco	advisory lock de sessão	`worker/__main__.py`
Entrega retomada após queda	lease em `next_attempt_at`	`alerts.py`
Histórico estável	paginação por cursor `(started_at, id)`	`queries.py`

6 Parte G — Testes, segurança e confiabilidade

6.1 Teste unitário e teste de integração

O PulseOps tem 251 testes em duas pastas:

Pasta	Quantos	O que testam	Precisam de banco?
`tests/unit/`	133	uma função ou classe isolada: regra de saúde, classificação de check, política SSRF, fórmulas	não
`tests/integration/`	118	várias peças juntas: API + serviços + PostgreSQL, worker + banco, migrações	sim, PostgreSQL de verdade

Um **teste unitário** é rápido e aponta exatamente onde está o erro. Exemplo, do arquivo `tests/unit/test_health.py`:

tests/unit/test_health.py, linhas 21–45

```
21 def run(
22     outcomes: list[Outcome], failure: int = 3, recovery: int = 2, state: TrackerState = INITIAL
23 ):
24     tracker = HealthTracker(Thresholds(failure, recovery))
25     decisions = []
26     for i, outcome in enumerate(outcomes):
27         obs = Observation(outcome, START + timedelta(minutes=i), f"resultado {i}")
28         decision = tracker.apply(state, obs)
29         decisions.append(decision)
30         state = decision.state
31     return state, decisions
32
33
34 F, S = Outcome.TIMEOUT, Outcome.HEALTHY
35
36
37 def actions(decisions):
38     return [d.incident_action for d in decisions]
39
40
41 def test_mandatory_example_fail_fail_success_opens_nothing() -> None:
42     state, decisions = run([F, F, S])
43     assert IncidentAction.OPEN not in actions(decisions)
44     assert (state.consecutive_failures, state.consecutive_successes) == (0, 1)
45     assert state.incident_open is False
```

- `run(...)` (linhas 21–32) é um ajudante: aplica uma lista de resultados à regra de saúde, um por minuto, e devolve o estado final e as decisões.
- `F, S` (linha 35) são apelidos curtos para “falha” (timeout) e “sucesso”.
- O teste das linhas 41–45 é o exemplo obrigatório “falha, falha, sucesso não abre incidente”. Ele não toca em banco nem em rede: só a regra pura de `src/pulseops/domain/health.py`.

Um **teste de integração** confere que as peças funcionam **juntas**. O mesmo exemplo é repetido com o banco em `test_threshold_example_with_database` (em `tests/integration/test_incident_lifecycle.py`): ali o check é gravado, o incidente é criado de verdade e as constraints do PostgreSQL estão valendo.

Os dois níveis se complementam. Se só houvesse testes unitários, um erro na transação (bug B6, por exemplo) passaria despercebido. Se só houvesse de integração, um erro na regra seria mais lento de achar.

6.2 Fixtures e dublês

Uma **fixture** do pytest é uma peça preparada antes do teste e entregue como parâmetro. Ficam em `tests/conftest.py`:

tests/conftest.py, linhas 51–71

```python
@pytest.fixture(scope="session")
def database_url() -> Iterator[str]:
    _recreate_database(TEST_DATABASE_URL)
    command.upgrade(alembic_config(TEST_DATABASE_URL), "head")
    yield TEST_DATABASE_URL


@pytest.fixture(scope="session")
async def engine(database_url: str) -> AsyncIterator[AsyncEngine]:
    engine = create_engine(database_url, pool_size=10)
    yield engine
    await engine.dispose()


@pytest.fixture
async def sessions(engine: AsyncEngine) -> SessionFactory:
    """Banco limpo para cada teste (TRUNCATE em todas as tabelas)."""
    tables = ", ".join(f'"{t.name}"' for t in Base.metadata.sorted_tables)
    async with engine.begin() as conn:
        await conn.execute(text(f"TRUNCATE {tables} RESTART IDENTITY CASCADE"))
    return create_session_factory(engine)
```

- `database_url` (escopo `session`): roda **uma vez por execução**. Apaga e recria o banco `pulseops_test` e aplica as migrações do Alembic. Assim, os testes usam o mesmo esquema da produção, não uma imitação.
- `engine`: uma conexão compartilhada por todos os testes.
- `sessions`: roda **antes de cada teste** e esvazia todas as tabelas com `TRUNCATE ... RESTART IDENTITY CASCADE`. Cada teste começa com o banco limpo e com os ids recomeçando em 1, então um teste não depende da ordem dos outros.

Um **dublê** (*test double*; "mock" é um tipo de dublê) substitui algo difícil de controlar. O PulseOps usa poucos, todos em `tests/support.py`:

tests/support.py, linhas 15–46

```python
class FakeClock:
    """Relógio parado que só anda quando o teste manda. Sem sleep nos testes."""

    def __init__(self, start: datetime = START) -> None:
        self._now = start
        self._mono = 1000.0

    def now(self) -> datetime:
        return self._now

    def monotonic(self) -> float:
        return self._mono

    def advance(self, seconds: float) -> None:
        self._now += timedelta(seconds=seconds)
```

```
30          self._mono += seconds
31
32
33  class FakeResolver:
34      """Resolve nomes conhecidos para IPs fixos; o resto vai para um IP público fictício."""
35
36      def __init__(self, mapping: dict[str, list[str]] | None = None) -> None:
37          self.mapping = mapping or {}
38          self.calls: list[str] = []
39
40      async def resolve(self, host: str, port: int) -> list[IPAddress]:
41          self.calls.append(host)
42          if host in self.mapping:
43              return [ipaddress.ip_address(a) for a in self.mapping[host]]
44          if host.endswith(".invalid"):
45              raise OSError("nome não resolvido")
46          return [ipaddress.ip_address("93.184.216.34")]
```

- `FakeClock` (linhas 15–30): um relógio parado que só anda quando o teste chama `advance`. O código de produção recebe um `Clock` como parâmetro (injeção de dependência, parte E), então o teste troca o relógio real por este. Resultado: nenhum teste usa `sleep`, e "esperar 3 intervalos" leva microssegundos.
- `FakeResolver` (linhas 33–46): responde a consultas DNS com IPs escolhidos pelo teste. É assim que os testes de SSRF simulam "o nome passou a apontar para 127.0.0.1" sem depender de um DNS de verdade.
- `ScriptedTarget` (linhas 49–85, no mesmo arquivo): um alvo HTTP simulado com o `httpx.MockTransport`. O teste escolhe o comportamento: `ok`, `slow`, `error`, `timeout`, `refused` ou `tls`.

O que não é dublê: o banco. Os testes de integração usam PostgreSQL de verdade.

6.3 Por que PostgreSQL de verdade, e não SQLite

Seria mais simples rodar os testes com SQLite, que é um arquivo e não precisa de servidor. Não daria para confiar no resultado, porque o PulseOps depende de recursos que o SQLite não tem ou faz diferente:

Recurso usado	No SQLite
`FOR UPDATE SKIP LOCKED`	não existe
`pg_try_advisory_lock`	não existe
`percentile_disc ... WITHIN GROUP`	não existe
`JSONB`	não existe (há JSON em texto)
`INSERT ... ON CONFLICT` com índice parcial	existe, com diferenças de sintaxe
`timestamptz`	não há tipo de data com fuso

Um teste que passa no SQLite e falha no PostgreSQL é pior do que nenhum teste: ele dá falsa confiança. O custo é exigir o container `pulseops-testdb` para rodar a suíte; o CI sobe um PostgreSQL como *service*.

6.4 Testes de regressão

Um **teste de regressão** é escrito para um bug que **aconteceu**, e falha na versão com o bug. Ele impede que o bug volte. O processo seguido no projeto, para cada bug do registro:

1. reproduzir o bug com um teste que falha;
2. corrigir o código;
3. ver o mesmo teste passar;
4. registrar em `docs/registro-de-bugs.md`: sintoma, como reproduzir, causa, correção, teste de regressão e limitação remanescente.

A regra de honestidade R4 proíbe o atalho de mudar o **resultado esperado** do teste para ele passar. Quando um teste falha, a pergunta é "o código está errado ou o teste está errado?", e a resposta precisa de um motivo. Nos bugs B4 e B5 o teste estava certo e o código foi corrigido.

6.4.1 Os dez bugs observados

Bug	Em uma frase	Lição
B1	o modelo ganhou uma coluna que a migração não tinha	o teste que compara modelos e migração pegou na hora
B2	`dict()` sobre um resultado do SQLAlchemy levantava `TypeError`	`dict(x)` trata qualquer objeto com `keys()` como mapeamento
B3	`Jinja2Templates(autoescape=True)` não existe no Starlette 1.x	conferir a API da versão instalada, não da memória
B4	hash corrompido com acento derrubava o login com erro 500	`UnicodeEncodeError` é um `ValueError`; tratar a família certa de exceções
B5	fuso inexistente virava `ZoneInfoNotFoundError` em vez de erro de validação	o Pydantic só converte `ValueError` e `AssertionError`
B6	check iniciado antes de pausar contava depois de retomar	qualquer evento que "recomeça a contagem" precisa invalidar execuções em voo
B7	nome de destino repetido no formulário gerava erro 500 (`MissingGreenlet`)	depois de `rollback()`, objetos do ORM expiram; guardar valores simples
B8	o worker atrasava checks quando havia mais vencidos que vagas	só um ensaio de carga mostrou; nenhum teste funcional pegaria
B9	a legenda do gráfico contradizia as métricas	uma revisão visual da tela pega o que os testes de dados não pegam
B10	a correção do B9 usou uma base de latência diferente da das métricas	conferir a definição no requisito antes de corrigir; explicar o código linha a linha achou o erro

B8 e B9 merecem atenção numa entrevista: ambos passaram por todos os testes existentes. B8 só apareceu medindo (ensaio de carga), e B9 só olhando a tela (instalação limpa). Isso mostra os limites de uma suíte de testes, por maior que seja.

6.5 Cobertura

Cobertura é a porcentagem de linhas do código executadas pelos testes. Medida no commit explicado (`docs/estrategia-de-testes.md`):

Parte	Meta	Medido
`src/pulseops/domain/`	≥ 95%	98%
`src/pulseops/security/`	≥ 95%	97%
pacote inteiro	≥ 90%	95%

Cobertura alta **não** prova que o código está certo: uma linha pode ser executada sem que nenhum `assert` confira o resultado dela. Ela só mostra o que **nunca** foi executado. A parte mais baixa é `worker/__main__.py` (44%), a montagem do processo do worker, exercitada pela demonstração e não pelos testes.

6.6 SSRF: um exemplo seguro

SSRF (*Server-Side Request Forgery*) é quando um atacante faz o **servidor** pedir uma URL que o atacante escolheu. Num monitor de APIs, isso é o produto funcionando: a operadora cadastra uma URL e o servidor a acessa. O risco é a URL apontar para dentro: `http://127.0.0.1:5432` (o banco), `http://10.0.0.5/admin` (a rede interna) ou `http://169.254.169.254/` (o serviço de metadados de nuvem, que pode entregar credenciais).

O PulseOps se defende em duas camadas (ADR 0005):

Camada 1 — no cadastro, o texto da URL:

`src/pulseops/security/url_policy.py, linhas 39–68`

```
39    def validate_url(self, url: str) -> Destination:
40        try:
41            parts = urlsplit(url.strip())
42            port = parts.port
43        except ValueError as exc:
44            raise UnsafeDestinationError("URL inválida") from exc
45
46        scheme = parts.scheme.lower()
47        if scheme not in ("http", "https"):
48            raise UnsafeDestinationError("use http:// ou https://")
49        if parts.username is not None or parts.password is not None:
50            raise UnsafeDestinationError("credenciais na URL não são permitidas")
51        host = (parts.hostname or "").lower()
52        if not host:
53            raise UnsafeDestinationError("URL sem host")
54        port = port or (443 if scheme == "https" else 80)
55
56        if self.is_demo_host(host, port):
57            return Destination(scheme, host, port)
58
59        if port not in self.allowed_ports:
60            allowed = ", ".join(str(p) for p in sorted(self.allowed_ports))
61            raise UnsafeDestinationError(f"porta {port} não permitida (permitidas: {allowed})")
62
63        literal = _parse_ip(host)
64        if literal is None and _NUMERIC_HOST.match(host):
65            raise UnsafeDestinationError("endereço IP em forma não canônica")
66        if literal is not None:
```

```
67            self.check_address(host, port, literal)
68        return Destination(scheme, host, port)
```

- linha 47: só `http` e `https` (nada de `file://`, `gopher://`);
- linha 49: nada de usuário e senha na URL;
- linha 59: só as portas 80, 443, 8080 e 8443;
- linha 64: recusa IPs escritos de forma estranha, como `2130706433` ou `0x7f.1`, que o sistema operacional entende como 127.0.0.1;
- linha 67: se o host for um IP, confere se é público.

Camada 2 — na conexão, o IP de verdade:

`src/pulseops/security/transport.py, linhas 44–68`

```python
44    async def resolve_allowed(self, host: str, port: int) -> IPAddress:
45        try:
46            addresses = await self._resolver.resolve(host, port)
47        except OSError as exc:
48            raise httpcore.ConnectError(f"não foi possível resolver o nome: {exc}") from exc
49        if not addresses:
50            raise httpcore.ConnectError("o nome não tem endereços")
51        # Se qualquer endereço for proibido, recusa: evita escolher "o IP bom" de uma
52        # resposta que mistura um IP público com um interno.
53        for address in addresses:
54            self._policy.check_address(host, port, address)
55        return addresses[0]
56
57    async def connect_tcp(
58        self,
59        host: str,
60        port: int,
61        timeout: float | None = None,  # noqa: ASYNC109 - assinatura do httpcore
62        local_address: str | None = None,
63        socket_options: Iterable[Any] | None = None,
64    ) -> httpcore.AsyncNetworkStream:
65        address = await self.resolve_allowed(host, port)
66        return await self._inner.connect_tcp(
67            str(address), port, timeout, local_address, socket_options
68        )
```

Esta é a camada que importa. Validar só no cadastro não basta, por causa do **DNS rebinding**: o nome `api.exemplo.com` aponta para um IP público no cadastro e, depois, o dono do domínio troca o registro para `127.0.0.1`. Por isso o PulseOps:

1. resolve o nome **no momento de cada conexão** (linha 46);
2. confere **todos** os IPs da resposta, e recusa se algum for proibido (linhas 53–54); escolher "o IP bom" de uma resposta mista seria uma brecha;
3. conecta **no IP que acabou de validar** (linhas 66–67), não no nome. Assim não existe uma segunda resolução, que poderia dar outra resposta.

O teste que prova a camada 2 (`tests/unit/test_transport.py`):

`tests/unit/test_transport.py, linhas 72–82`

```python
72 async def test_rebinding_is_checked_at_connect_time() -> None:
73    """O nome era público no cadastro e passa a apontar para a rede interna."""
74    resolver = FakeResolver({"rebind.example.com": ["93.184.216.34"]})
75    await validate_destination(DestinationPolicy(), "http://rebind.example.com/", resolver)
76
77    resolver.mapping["rebind.example.com"] = ["127.0.0.1"]
78    inner = RecordingBackend()
79    async with guarded_client(resolver, inner) as client:
```

```
        with pytest.raises(UnsafeDestinationError, match="loopback"):
            await client.get("http://rebind.example.com/")
    assert inner.connections == []  # nenhuma conexão foi aberta
```

O nome passa no cadastro apontando para um IP público. Depois, o teste troca a resposta do DNS falso para `127.0.0.1`. A requisição é recusada com "loopback" e `inner.connections == []` prova que **nenhuma conexão foi aberta**.

O laboratório de falhas precisa acessar `lab-target`, que é um IP privado da rede do Docker. Para isso existe uma lista de exceções **por nome e porta** (`PULSEOPS_DEMO_ALLOWED_HOSTS`), que só vale com `PULSEOPS_DEMO_MODE=true`.

Limites honestos: redirecionamentos não são seguidos (um 302 é tratado como "status inesperado"), o que evita a volta pela porta dos fundos, mas também quer dizer que monitores de URLs que redirecionam precisam apontar para o destino final. O modelo de ameaças (`docs/modelo-de-ameacas.md`) lista os riscos que continuam.

6.7 Autenticação, sessão e CSRF

Autenticação responde "quem é você?". O PulseOps tem uma operadora, criada pela linha de comando, e login por formulário (ADR 0008).

- **Senha:** guardada como hash **Argon2id** (`src/pulseops/security/passwords.py`). Um hash é uma transformação de mão única: dá para conferir se uma senha bate, mas não dá para recuperar a senha a partir do hash. O Argon2id é lento de propósito, para que testar bilhões de senhas seja caro.
- **Mesmo tempo para usuário inexistente:**

src/pulseops/services/auth.py, linhas 33–38

```
async def authenticate(session: AsyncSession, username: str, password: str) -> Operator | None:
    operator = await session.scalar(select(Operator).where(Operator.username == username.strip()))
    # verify_password roda mesmo sem usuário, para não revelar quais usuários existem pelo tempo.
    if verify_password(operator.password_hash if operator else None, password):
        return operator
    return None
```

`verify_password` roda mesmo quando o usuário não existe (com `None`). Sem isso, a resposta para um usuário inexistente seria mais rápida, e um atacante poderia descobrir quais usuários existem medindo o tempo.

- **Sessão:** depois do login, o servidor sorteia um token aleatório de 32 bytes (`secrets.token_urlsafe`) e o envia num **cookie**. O banco guarda só o **SHA-256 do token**:

src/pulseops/services/auth.py, linhas 48–65

```
async def start_session(
    session: AsyncSession, operator: Operator, *, now: datetime, ttl: timedelta
) -> NewSession:
    token = secrets.token_urlsafe(32)
    csrf_token = secrets.token_urlsafe(32)
    expires_at = now + ttl
    session.add(
        OperatorSession(
            token_hash=hash_token(token),
            operator_id=operator.id,
            csrf_token=csrf_token,
```

```
            created_at=now,
            expires_at=expires_at,
        )
    )
    # Aproveita o login para limpar sessões vencidas.
    await session.execute(delete(OperatorSession).where(OperatorSession.expires_at <= now))
    return NewSession(token, csrf_token, expires_at)
```

Se alguém copiar a tabela `operator_sessions`, não consegue usar as sessões. O teste `test_session_token_is_stored_hashed` (em `tests/integration/test_auth.py`) confere isso.

- **Cookie:** `HttpOnly` (o JavaScript da página não lê), `SameSite=Lax` (o navegador não o envia em formulários `POST` vindos de outro site, mas envia ao seguir um link para o painel, o que mantém a operadora logada) e `Secure` quando configurado (só vai por HTTPS). O cookie temporário do formulário de login é `SameSite=Strict`. Teste: `test_session_cookie_flags`.

CSRF (*Cross-Site Request Forgery*): um site malicioso faz o navegador da operadora, que está logada, enviar um formulário para o PulseOps, por exemplo "arquivar monitor 3". O cookie iria junto automaticamente. A defesa: cada sessão tem um segundo token, o **token CSRF**, que precisa vir no corpo do formulário (campo `csrf_token`) ou no header `X-CSRF-Token` da API. O site malicioso não consegue ler esse valor, então não consegue montar o pedido.

`src/pulseops/web/deps.py, linhas 80–95`

```
def tokens_match(expected: str | None, received: str | None) -> bool:
    if not expected or not received:
        return False
    return hmac.compare_digest(expected, received)


async def require_api_auth(request: Request, ctx: MaybeAuth) -> AuthContext:
    """API JSON: 401 sem sessão; métodos que alteram dados exigem o header X-CSRF-Token."""
    if ctx is None:
        raise HTTPException(status_code=401, detail="autenticação necessária")
    if request.method in UNSAFE_METHODS and not tokens_match(
        ctx.csrf_token, request.headers.get("X-CSRF-Token")
    ):
        raise HTTPException(status_code=403, detail="token CSRF ausente ou inválido")
    return ctx
```

`hmac.compare_digest` compara os tokens em tempo constante, sem parar no primeiro caractere diferente. Testes: `test_api_mutation_requires_csrf_header` e `test_form_action_requires_csrf_field`.

Limites honestos: não há limite de tentativas de login nem bloqueio após erros, não há segundo fator e não há usuários com papéis diferentes.

6.8 Retentativas e duplicidade

O envio de alertas segue o padrão **outbox** (ADR 0007):

1. Na **mesma transação** que abre o incidente, o PulseOps grava o evento (`alert_events`) e uma entrega por destino ativo (`deliveries`).
2. Um laço separado do worker reserva entregas vencidas e faz o POST do webhook.
3. O resultado é gravado em `delivery_attempts`, uma linha por tentativa.

Por que não enviar direto, na hora de abrir o incidente? Porque o envio pode falhar ou demorar. Se fosse dentro da transação, um destino lento seguraria o banco; se fosse depois do commit, uma queda entre o commit e o envio perderia o alerta para sempre.

Retentativas com espera exponencial:

src/pulseops/services/alerts.py, linhas 92–99

```python
def backoff_delay(attempt: int, base_seconds: float, max_seconds: float) -> float:
    """Espera antes da próxima tentativa após `attempt` falhas: base × 2^(attempt−1), com teto."""
    return float(min(base_seconds * 2 ** (attempt - 1), max_seconds))


def is_permanent_status(status_code: int) -> bool:
    """4xx indica pedido recusado pelo destino; repetir não ajuda. 408 e 429 são temporários."""
    return 400 <= status_code < 500 and status_code not in (408, 429)
```

Depois da 1ª falha, espera `base`; depois da 2ª, `2 × base`; depois da 3ª, `4 × base`; e assim por diante, até o teto. Com os padrões (5 s e teto de 300 s), as esperas são 5 s, 10 s, 20 s, 40 s e 80 s; se a 6ª tentativa (o máximo padrão) também falhar, a entrega fica `failed`. O teto só entra em jogo com mais tentativas configuradas. Esperar cada vez mais dá tempo para o destino se recuperar sem ser bombardeado. Erros 4xx (exceto 408 e 429) são **permanentes**: o destino recusou o pedido, e repetir não muda nada.

Duplicidade: a garantia é **pelo menos uma vez**, não "exatamente uma vez". Se o worker morrer depois de o destino receber o POST e antes de gravar o sucesso, a reserva vence e o envio é repetido. Por isso cada evento leva um `event_id` estável (`incident:42:opened`), e o destino deve ignorar ids repetidos. O receptor do laboratório faz isso, e o passo 11 da demonstração confere. Teste: `test_expired_lease_is_retried_which_may_duplicate` (o nome já admite a duplicata).

"Exatamente uma vez" entre dois sistemas que não compartilham transação não é possível de garantir só de um lado. Dizer isso numa entrevista mostra que você entende o problema.

6.9 Limitações conhecidas

Coletadas do README, do modelo de ameaças e de `docs/operacao.md`:

Limitação	Consequência
Um worker só	não há alta disponibilidade; se a máquina cair, ninguém é avisado
Tick de 1 s	um check pode começar até cerca de 1 s depois do horário previsto
Um operador, sem papéis	não serve para uma equipe com permissões diferentes
Sem limite de tentativas de login	uma senha fraca pode ser descoberta por tentativa e erro
Webhook sem assinatura HMAC	o destino não consegue provar que o alerta veio do PulseOps
Sem autenticação nos alvos	não monitora APIs que exigem token
Redirecionamentos não seguidos	URL que redireciona aparece como "status inesperado"

Limitação	Consequência
Uma migração só	evolução de esquema com dados nunca foi exercitada
Ensaio de carga local	200 monitores numa máquina; não é teste de capacidade em produção
`Result.tuples()` obsoleto	o SQLAlchemy 2.1 avisa que o método está obsoleto (aviso nos testes); funciona, mas precisa ser trocado antes de uma versão que o remova

7 Parte H — Perguntas de entrevista

São 34 perguntas específicas sobre o PulseOps. As 15 primeiras são as obrigatórias da especificação, na mesma ordem. Cada uma tem:

- **Resposta curta**: o que dizer em 20 a 40 segundos;
- **Explicação**: o raciocínio, para você entender e não decorar;
- **Onde ver**: arquivo, trecho ou teste para mostrar ou reler;
- **Aprofundamento provável**: a pergunta que costuma vir depois, com uma pista;
- **Limite honesto**: até onde você pode afirmar com segurança.

Uma regra geral para todas: o projeto foi escrito com o Claude Code a partir da sua especificação (README, seção "Como este projeto foi feito"). Diga isso se perguntarem, e responda sobre o código como alguém que o estudou e consegue explicá-lo, não como quem o digitou linha por linha.

7.1 H1. Por que FastAPI?

Resposta curta. A especificação pedia uma API JSON e um painel no mesmo processo, com validação de entrada forte e código assíncrono. O FastAPI dá as três coisas: rotas assíncronas, validação com Pydantic a partir dos tipos e injeção de dependências pelo `Depends`.

Explicação. O worker e a API compartilham os mesmos serviços assíncronos (`services/*.py`, com `AsyncSession`). Um framework síncrono obrigaria a duplicar o acesso a banco ou a rodar código assíncrono dentro de threads. O Pydantic faz a validação de formato (`web/schemas.py`, por exemplo `extra="forbid"` recusa campos desconhecidos) e devolve 422 com a lista de erros. As dependências (`Db`, `State`, `ApiAuth` em `web/deps.py`) deixam as rotas curtas e trocáveis nos testes.

Onde ver. `src/pulseops/web/api.py`, `src/pulseops/web/deps.py`, `src/pulseops/web/schemas.py`; teste `test_unknown_fields_and_methods_rejected`.

Aprofundamento provável. "E Django?" Pista: Django traz ORM, admin e autenticação prontos, mas o ORM dele é outro (a especificação pedia SQLAlchemy e Alembic) e o suporte assíncrono no acesso a banco é mais recente. Para um serviço pequeno de API, FastAPI + SQLAlchemy é uma combinação comum nas vagas pesquisadas.

Limite honesto. A escolha da stack veio da especificação, não de uma comparação feita por você em produção. Não diga que o FastAPI é "mais rápido" sem ter medido.

7.2 H2. Onde há orientação a objetos?

Resposta curta. Em quatro formas: classes com estado e comportamento (`Worker`, `HttpChecker`, `WebhookNotifier`, `HealthTracker`); herança onde uma biblioteca exige (`GuardedNetworkBackend` herda de `httpcore.AsyncNetworkBackend`, os modelos herdam de `Base`); protocolos (`Clock`, `Checker`, `Notifier`, `Resolver`); e objetos de valor imutáveis com `@dataclass(frozen=True)` (`ClaimedRun`, `Thresholds`, `Decision`).

Explicação. `Worker` guarda as vagas em andamento (`_inflight`) e expõe operações (`dispatch_checks`, `run`). `HealthTracker` encapsula a regra de saúde: recebe o estado anterior e uma observação e devolve uma `Decision`, sem efeito colateral. Herança é usada pouco e só quando a biblioteca pede um tipo base. O resto é composição.

Onde ver. `src/pulseops/worker/runner.py` (linhas 33–54), `src/pulseops/domain/health.py`, `src/pulseops/security/transport.py` (linha 33), `src/pulseops/clock.py`.

Aprofundamento provável. "O que é um Protocol?" Pista: é tipagem estrutural. Qualquer classe com o método `now()` e `monotonic()` serve como `Clock`, sem herdar de nada. É o que permite passar o `FakeClock` nos testes.

Limite honesto. Não há padrões elaborados (fábricas abstratas, repositórios genéricos). Não diga que o projeto "aplica todos os princípios SOLID".

7.3 H3. Por que composição?

Resposta curta. O `Worker` não cria o verificador, o notificador nem o relógio: recebe prontos no construtor. Assim, cada peça pode ser trocada (nos testes, por dublês) e testada separadamente.

Explicação. Com herança, um `Worker` que precisasse de outro tipo de verificador viraria uma subclasse nova. Com composição, basta passar outro objeto que siga o protocolo `Checker`. Os testes passam um `HttpChecker` com `httpx.MockTransport` e um `FakeClock`. A montagem real fica num lugar só: `src/pulseops/worker/__main__.py`.

Onde ver. `src/pulseops/worker/runner.py` linhas 34–48; `src/pulseops/worker/__main__.py` (montagem); `tests/support.py` (`ScriptedTarget`).

Aprofundamento provável. "Isso é injeção de dependências?" Pista: sim, feita à mão, pelo construtor. Na API, o FastAPI faz o mesmo com `Depends`.

Limite honesto. Não há contêiner de injeção de dependências. É composição simples, o que é adequado ao tamanho do projeto.

7.4 H4. O que acontece quando o worker reinicia?

Resposta curta. Nada se perde, porque o estado mora no banco. Ao subir, o worker pega o advisory lock, volta a reservar execuções vencidas, marca como `internal_error` os checks que ficaram `running` e registra uma **lacuna de cobertura** se ficou mais de um intervalo sem verificar. Um incidente aberto continua aberto e a contagem segue.

Explicação. Os contadores (`consecutive_failures`…) ficam na tabela `monitors`, não na memória. `mark_abandoned_runs` resolve os checks interrompidos sem contar falha do alvo (não foi o alvo que falhou, foi o worker). Em `_reserve`, se `now - scheduled_at > interval`, o PulseOps não tenta "recuperar" todos os checks perdidos (seria uma avalanche): grava um `CoverageGap` e agenda para agora. Entregas que estavam `sending` voltam a ser elegíveis quando a reserva vence.

Onde ver. `src/pulseops/services/processing.py` linhas 88–103 e 239–264; testes `test_restart_with_open_incident_continues_and_records_gap` e `test_interrupted_run_becomes_internal_error_not_failure`.

Aprofundamento provável. "E se dois workers subirem?" Pista: H16 e o `pg_try_advisory_lock`; o segundo sai com código 1 (`test_second_worker_is_refused`).

Limite honesto. Reinício foi testado com relógio simulado e na demonstração com Docker. Não houve teste com o processo morto à força (`kill -9`) no meio de uma gravação, embora a transação garanta que ela não fique pela metade.

7.5 H5. Como impedir incidentes duplicados?

Resposta curta. Em duas camadas: a regra só manda **abrir** quando não há incidente aberto (senão manda **atualizar**), e o banco tem um índice único parcial que proíbe um segundo incidente `open` para o mesmo monitor.

Explicação. `record_result` trava o monitor (`FOR UPDATE`) e lê o incidente aberto também com `FOR UPDATE`, então duas gravações simultâneas do mesmo monitor acontecem uma depois da outra. Mesmo que um bug escapasse disso, o índice `uq_incidents_one_open_per_monitor` faria o `INSERT` falhar, a transação inteira seria desfeita e nenhum alerta duplicado sairia.

Onde ver. `src/pulseops/db/models.py` linhas 182–188; `src/pulseops/services/processing.py` linhas 149 e 184; testes `test_database_rejects_second_open_incident` e `test_mandatory_example_more_failures_update_the_same_incident`.

Aprofundamento provável. "E alerta duplicado?" Pista: `alert_events.event_key` é único (`incident:{id}:opened`), e há uma entrega por par (evento, destino).

Limite honesto. "Sem duplicatas nos cenários testados" é o que dá para afirmar. Não houve teste de carga concorrente com vários processos gravando ao mesmo tempo, porque o projeto só admite um worker.

7.6 H6. Por que uma sessão SQLAlchemy não deve ser compartilhada entre tarefas?

Resposta curta. Uma `AsyncSession` representa **uma** conversa com o banco, em uma conexão e uma transação. Ela não é segura para uso simultâneo: duas tarefas `asyncio` usando a mesma sessão misturariam comandos na mesma transação e o SQLAlchemy levanta erro. Cada unidade de trabalho abre a sua.

Explicação. No worker, cada check roda numa tarefa própria (`asyncio.create_task(self._run_one(run))`), e `claim_due_runs` e `record_result` recebem a **fábrica** de sessões (`SessionFactory`), não uma sessão. Cada chamada faz `async with sessions.begin() as session:`, com começo e fim claros. Além da segurança, isso mantém as transações curtas: se uma tarefa segurasse a sessão durante a requisição HTTP, seguraria travas.

Onde ver. `src/pulseops/worker/runner.py` linhas 67–75 e 102–122; `src/pulseops/services/processing.py` (todas as funções recebem `sessions: SessionFactory`); `src/pulseops/db/session.py`.

Aprofundamento provável. "E na API?" Pista: `get_db` abre uma sessão por requisição HTTP, e uma requisição é tratada por uma tarefa só.

Limite honesto. O PulseOps nunca tentou compartilhar sessões, então você não viu o erro na prática. Fale do motivo, não de uma experiência.

7.7 H7. Por que não executar tudo na rota?

Resposta curta. Porque checks são periódicos e alertas precisam de retentativas: isso precisa continuar mesmo sem ninguém acessar a API. A rota só grava a intenção (por exemplo, "verificar agora" põe `check_requested = true`), e o worker faz o trabalho.

Explicação. Se a rota fizesse a requisição HTTP ao alvo, a resposta da API demoraria até o timeout do alvo (30 s). Se fizesse com `BackgroundTasks` do FastAPI, o trabalho morreria com o processo e não teria retentativa (a especificação proíbe isso para trabalho durável). Separar também isola falhas: um alvo lento não deixa o painel lento.

Onde ver. `src/pulseops/services/monitors.py` (`request_check_now`, linhas 228–236); `docs/decisoes/0002-api-e-worker-separados.md`; teste `test_check_now_marks_next_check_as_manual`.

Aprofundamento provável. "Como a API sabe que o worker está vivo?" Pista: H21, heartbeat.

Limite honesto. A API e o worker compartilham o banco; não há fila de mensagens. Para este volume, o banco como fila é suficiente, e você pode dizer por quê (H14).

7.8 H8. Como funciona a transação?

Resposta curta. `async with sessions.begin() as session:` abre a transação; se o bloco termina normalmente, faz `COMMIT`; se uma exceção escapa, faz `ROLLBACK`. Tudo que está dentro vale junto ou não vale.

Explicação. O exemplo central é `record_result`: concluir o check, atualizar os contadores, abrir o incidente, gravar a mudança de saúde e criar o alerta e as entregas acontecem numa transação só. Se o banco cair no meio, nada fica pela metade, e o check continua `running` até a manutenção marcá-lo como interrompido. A requisição HTTP fica **fora** de qualquer transação (ADR 0003).

Onde ver. Parte F deste guia; `src/pulseops/services/processing.py` linhas 139–178; testes `test_outbox_event_is_atomic_with_incident` e `test_database_failure_while_recording_does_not_count_as_target_failure`.

Aprofundamento provável. "Qual nível de isolamento?" Pista: o padrão do PostgreSQL, *Read Committed*. O PulseOps não depende de um nível mais forte porque trava explicitamente as linhas que disputa (`FOR UPDATE`).

Limite honesto. O isolamento não foi alterado nem testado em outros níveis.

7.9 H9. Qual a diferença entre lentidão e falha?

Resposta curta. Lento é quando o alvo respondeu com um status aceito, mas acima do limite de latência. Falha é quando não respondeu certo: timeout, erro de conexão, erro TLS ou status fora da lista. Lento deixa o monitor **degradado**, mas não conta para abrir incidente; falha conta.

Explicação. Se lentidão abrisse incidente, um pico de carga geraria alertas de "fora do ar" para um serviço no ar. O PulseOps também conta uma resposta lenta como **sucesso** para a recuperação: um serviço que voltou, mas lento, está disponível. O timeout é falha porque, para quem usa, "não respondeu".

Onde ver. `src/pulseops/domain/outcomes.py`; `docs/decisoes/0004-lentidao-nao-abre-incidente.md`; testes `test_slow_is_degraded_but_not_a_failure` e `test_slow_response_counts_towards_recovery`.

Aprofundamento provável. "Como mede o tempo?" Pista: com o relógio **monotônico** (`clock.monotonic()`), que não volta para trás quando o relógio do sistema é ajustado.

Limite honesto. O tempo medido inclui DNS, conexão e TLS, porque é o que o usuário sente. Não é só o tempo de processamento do servidor.

7.10 H10. Como calcular métricas sem dados?

Resposta curta. Mostrando "N/A", nunca 0% nem 100%. Sem checks na janela, a proporção não tem denominador; inventar um número seria mentir.

Explicação. `ratio(part, total)` devolve `None` quando `total <= 0`, e o painel mostra `None` como "N/A". O denominador também é escolhido com cuidado: checks `internal_error` (falha do próprio PulseOps) saem da conta, porque não dizem nada sobre o alvo. Latência média e p95 usam só checks que receberam resposta HTTP (`healthy`, `slow` e `unexpected_status`); timeout e erro de conexão ficam fora. Lacunas de cobertura aparecem ao lado das métricas.

Onde ver. `src/pulseops/domain/metrics.py` linhas 22–26; `src/pulseops/services/queries.py` (`monitor_metrics`); testes `test_no_data_returns_na_not_zero_or_hundred` e `test_paused_period_produces_no_samples`.

Aprofundamento provável. "Disponibilidade de 99,9% com 3 checks?" Pista: o painel mostra o número de amostras junto da proporção; poucas amostras não sustentam conclusão nenhuma.

Limite honesto. As métricas são proporções de **checks**, não de tempo. Um intervalo de 60 s não vê uma queda de 20 s entre dois checks.

7.11 H11. O que ocorre se o webhook cair?

Resposta curta. As verificações continuam normalmente. A entrega falha, é registrada como tentativa e volta para a fila com espera exponencial, até dar certo ou atingir o máximo de tentativas.

Explicação. Alertas são entregues por outro laço do worker (`run_deliveries_once`), em tarefas separadas, com timeout próprio. Um destino lento ou fora do ar não ocupa as vagas dos checks. Cada tentativa vira uma linha em `delivery_attempts`, visível no painel de notificações. Os passos 9, 10 e 11 da

demonstração mostram exatamente isso: receptor fora, checks continuando, e a entrega acontecendo quando o receptor volta.

Onde ver. `src/pulseops/worker/runner.py` linhas 126–159; `src/pulseops/services/alerts.py`; testes `test_slow_webhook_does_not_block_checks` e `test_retries_with_capped_exponential_backoff`.

Aprofundamento provável. "E se ficar fora para sempre?" Pista: depois de `webhook_max_attempts` (6 por padrão) a entrega fica `failed`, e isso aparece no painel. Não há segundo canal de alerta.

Limite honesto. Não existe alerta sobre "o alerta falhou". A operadora só vê olhando o painel.

7.12 H12. Por que a entrega pode se repetir?

Resposta curta. Porque a garantia é "pelo menos uma vez". Se o worker morrer depois de o destino receber o POST e antes de gravar o sucesso, a reserva vence e o envio é feito de novo. O `event_id` estável permite ao destino descartar a repetição.

Explicação. Não há transação que englobe o banco do PulseOps e o servidor do destino. Um dos dois lados sempre pode falhar entre "enviei" e "anotei que enviei". Escolher "no máximo uma vez" perderia alertas; escolher "pelo menos uma vez" e deduplicar no destino é o padrão usado por serviços de webhook.

Onde ver. `src/pulseops/services/alerts.py` linhas 111–143 (reserva com lease); `lab/receiver.py` (deduplicação no receptor do laboratório); teste `test_expired_lease_is_retried_which_may_duplicate`; RF06.8 em `docs/requisitos.md`.

Aprofundamento provável. "Como evitar que uma gravação atrasada sobrescreva uma tentativa nova?" Pista: o `UPDATE` só vale se `status = 'sending'` e `attempt_count` for o da própria tentativa (parte F).

Limite honesto. A deduplicação depende do destino. O PulseOps não pode garanti-la sozinho.

7.13 H13. Como o projeto trata SSRF?

Resposta curta. Valida o texto da URL no cadastro (esquema, porta, credenciais, formas estranhas de IP) e, principalmente, valida o IP **na hora de cada conexão**, conectando no mesmo IP validado. Isso fecha a porta do DNS rebinding. Redirecionamentos não são seguidos e o proxy do ambiente é ignorado.

Explicação. Ver a parte G. O ponto que costuma impressionar é o `GuardedNetworkBackend`: o HTTPX chama `connect_tcp(host, porta)`, o PulseOps resolve o nome, recusa se **qualquer** IP for proibido e conecta no IP. O header `Host` e o SNI continuam com o nome original, então HTTPS funciona.

Onde ver. `src/pulseops/security/url_policy.py`, `src/pulseops/security/transport.py`; testes `test_rebinding_is_checked_at_connect_time`, `test_mixed_public_and_private_answers_are_refused`, `test_blocked_urls`.

Aprofundamento provável. "E se o HTTPX mudar por dentro?" Pista: o projeto troca o pool interno (`transport._pool`), um detalhe interno da versão 0.28. O teste `test_connects_to_validated_ip_and_keeps_host_header` quebra se isso mudar; está no modelo de

ameaças.

Limite honesto. A proteção está na aplicação. Não há firewall de saída configurado pelo projeto, que seria a segunda barreira numa instalação séria.

7.14 H14. Qual a limitação de escala?

Resposta curta. Um worker por banco, com até 20 checks simultâneos por padrão e tick de 1 s. Foi medido: 200 monitores a cada 10 s numa máquina de 4 CPUs, com 99,5% dos checks esperados feitos e atraso mediano de 0,64 s.

Explicação. O advisory lock impede um segundo worker de propósito: assim não é preciso coordenar quem verifica o quê. Para escalar, seria preciso permitir vários workers (a reserva com `SKIP LOCKED` já está pronta para isso, mas o heartbeat, a manutenção e a retenção supõem um só) e, mais tarde, particionar a tabela `checks` por data.

Onde ver. `docs/evidencias/ensaio-de-carga.md`; `src/pulseops/worker/__main__.py` linhas 24–32; ADR 0002.

Aprofundamento provável. "Qual seria o gargalo primeiro?" Pista: com alvos lentos, as vagas de concorrência (cada check ocupa uma vaga até o timeout). Com muitos monitores, o volume de linhas em `checks`.

Limite honesto. Não houve teste com mais de 200 monitores nem com alvos reais na internet (o ensaio usou o alvo do laboratório). Não fale em "milhares de monitores".

7.15 H15. O que você mudaria numa versão comercial?

Resposta curta. Vários workers com alta disponibilidade, contas e papéis, limite de tentativas de login, assinatura HMAC nos webhooks, mais canais de alerta (e-mail, Slack), autenticação nos alvos, verificações de várias regiões e um firewall de saída.

Explicação. Cada item vem de uma limitação documentada (parte G). A ordem que você pode defender: primeiro segurança (limite de login, HMAC), porque são riscos já identificados; depois disponibilidade (vários workers), porque hoje uma queda da máquina silencia tudo; por último funcionalidades.

Onde ver. README, seção de limitações; `docs/modelo-de-ameacas.md`, seção "Limitações remanescentes".

Aprofundamento provável. "Como você monitoraria o próprio monitor?" Pista: um *dead man's switch* externo (o modelo passivo do Healthchecks): o PulseOps manda um sinal periódico para fora, e o silêncio vira alerta.

Limite honesto. Nada disso está implementado. É roadmap, e deve ser dito como roadmap.

7.16 H16. Por que API e worker em processos separados?

Resposta curta. Para que um não derrube o outro. Se o painel estiver sob carga, os checks continuam; se o worker travar, o painel mostra "worker sem sinal". Cada processo pode ser reiniciado sozinho.

Explicação. No `compose.yaml` são dois serviços com a mesma imagem e comandos diferentes. A comunicação é só pelo banco. Por isso a API consegue mostrar que o worker parou: ela lê o último heartbeat.

Onde ver. `compose.yaml`; `docs/decisoes/0002-api-e-worker-separados.md`; teste `test_stopped_worker_makes_data_stale`.

Aprofundamento provável. "Por que não Celery?" Pista: Celery exige um broker (Redis ou RabbitMQ), mais uma peça para operar. Para um worker e agendamento simples, o banco como fila resolve.

Limite honesto. Separar processos não dá alta disponibilidade: os dois rodam na mesma máquina.

7.17 H17. Por que async e não threads?

Resposta curta. O trabalho do worker é quase todo **espera** (rede e banco). Com `asyncio`, uma thread só alterna entre centenas de esperas sem o custo de criar uma thread para cada uma.

Explicação. Cada check é uma corrotina que passa quase todo o tempo em `await` (conexão, resposta, gravação). O event loop executa outra enquanto isso. O limite de simultaneidade é explícito: no máximo `worker_concurrency` tarefas em `_inflight`. A resolução DNS (`getaddrinfo`), que é bloqueante, vai para uma thread do próprio loop (`loop.getaddrinfo`).

Onde ver. `src/pulseops/worker/runner.py`; `src/pulseops/security/transport.py` linhas 25–30; ADR 0003.

Aprofundamento provável. "O que acontece se alguém escrever código bloqueante numa corrotina?" Pista: trava o loop inteiro, todos os checks param. Por isso o corpo da resposta não é lido e nada pesado roda no laço.

Limite honesto. Async não ajuda trabalho de CPU. O PulseOps não tem esse tipo de trabalho, então não houve necessidade de processos extras.

7.18 H18. Como o worker sabe o que verificar e quando?

Resposta curta. Cada monitor tem `next_run_at`. A cada tick, o worker reserva os monitores ativos com `next_run_at <= agora` e sem check em andamento, grava o check como `running` e avança `next_run_at` em um intervalo.

Explicação. O agendamento fica no banco, não num temporizador em memória. Por isso sobrevive a reinícios. O índice parcial `ix_monitors_due` torna a consulta barata. O novo `next_run_at` é calculado a partir do horário **agendado**, não do horário em que o check terminou, para o ritmo não "escorregar".

Onde ver. `src/pulseops/services/processing.py` linhas 50–136; parte F, "Reservar monitores vencidos"; teste `test_scheduled_time_and_observed_time_are_both_recorded`.

Aprofundamento provável. “E se o check demorar mais que o intervalo?” Pista: não pode, por constraint (`timeout_seconds < interval_seconds`), e a reserva pula monitores com check em andamento.

Limite honesto. A precisão é de cerca de 1 s (o tick).

7.19 H19. O que é o run_key?

Resposta curta. A identidade de uma execução agendada: `"{monitor_id}:{horário agendado}"`. É única no banco, então a mesma execução não vira dois checks, mesmo que seja reservada duas vezes.

Explicação. O `INSERT ... ON CONFLICT (run_key) DO NOTHING RETURNING id` insere ou não faz nada; se não inseriu, não volta id, e o worker não executa. É uma forma de **idempotência**: repetir a operação dá o mesmo resultado que fazê-la uma vez.

Onde ver. `src/pulseops/services/processing.py` linhas 46–47 e 105–123; teste `test_same_run_identity_cannot_be_reserved_twice`.

Aprofundamento provável. “E gravar o resultado duas vezes?” Pista: `record_result` só grava se o check ainda está `running`; a segunda chamada devolve `False` (`test_reprocessed_result_is_ignored`).

Limite honesto. Dois cliques em “verificar agora” antes de o worker reservar viram um único check, porque cada clique só antecipa `next_run_at`. Isso evita fila de checks manuais, mas não foi pedido explicitamente pela especificação.

7.20 H20. Como impedir que um check comece antes do anterior terminar?

Resposta curta. A reserva pula monitores que já têm um check `running`, e o banco tem um índice único parcial que proíbe dois checks `running` no mesmo monitor.

Explicação. É o mesmo padrão de H5 aplicado aos checks. O “verificar agora” também passa por esse controle: se houver um check em andamento, o manual espera ele terminar (CB5).

Onde ver. `src/pulseops/db/models.py` linhas 147–153; testes `test_overlapping_check_is_not_started` e `test_database_rejects_second_running_check`.

Aprofundamento provável. “Por que as duas coisas?” Pista: a consulta evita o trabalho; o índice garante a regra mesmo se a consulta tiver um bug.

Limite honesto. Nada a ressalvar além do que já foi dito: é garantido pelo banco.

7.21 H21. Como o painel mostra que o worker parou?

Resposta curta. O worker grava um heartbeat periódico. Se o último for mais antigo que o limite, o painel mostra “Worker sem sinal” e marca todos os monitores ativos como **desatualizados**. Um monitor também fica desatualizado se o último check for velho demais, mesmo com o worker vivo.

Explicação. Sem isso, um worker parado deixaria o painel congelado em “saudável”, o pior tipo de erro para um monitor. O limite por monitor é `2 × intervalo + timeout + folga`.

Onde ver. `src/pulseops/services/queries.py` linhas 68–87 (`worker_status`, `is_stale`); `src/pulseops/worker/runner.py` linhas 163–176; testes `test_stopped_worker_makes_data_stale` e `test_old_last_check_is_stale_even_with_worker_alive`.

Aprofundamento provável. "E se o banco cair?" Pista: a API responde 503 em `/health/ready`, e o painel não carrega (`test_ready_returns_503_when_database_is_down`).

Limite honesto. Ninguém é **avisado** de que o worker parou; só quem abre o painel vê.

7.22 H22. Como testar tempo sem `sleep`?

Resposta curta. O código recebe um `Clock` e os testes passam um `FakeClock`, que só anda quando o teste manda. "Esperar 3 intervalos" vira `clock.advance(...)`.

Explicação. Testes com `sleep` são lentos e instáveis (passam ou falham dependendo da carga da máquina). Com o relógio injetado, os 251 testes rodam em menos de um minuto e dão sempre o mesmo resultado. A lentidão do alvo simulado também é feita avançando o relógio.

Onde ver. `src/pulseops/clock.py`; `tests/support.py` linhas 15–30.

Aprofundamento provável. "Algum teste usa tempo real?" Pista: o teste do B8 (`test_dispatcher_refills_freed_slots_without_waiting_a_tick`) mede com tempo real, porque o bug era justamente de espera.

Limite honesto. O relógio falso não pega problemas de fuso ou de ajuste de relógio do sistema; para isso há os testes de fuso horário e o uso do relógio monotônico.

7.23 H23. Por que PostgreSQL de verdade nos testes?

Resposta curta. Porque as garantias mais importantes estão no banco: `SKIP LOCKED`, advisory lock, índices parciais, `percentile_disc`, `JSONB`. O SQLite não tem a maioria disso; um teste nele não provaria nada.

Explicação. Ver a tabela da parte G. O custo é precisar do container `pulseops-testdb`; o CI sobe o PostgreSQL 16 como serviço.

Onde ver. `tests/conftest.py`; `.github/workflows/ci.yml`.

Aprofundamento provável. "Como isolar os testes?" Pista: `TRUNCATE` de todas as tabelas antes de cada teste.

Limite honesto. Os testes não rodam em paralelo (`pytest -n`), porque compartilham o mesmo banco.

7.24 H24. Como a senha é guardada?

Resposta curta. Como hash Argon2id, com os parâmetros padrão da biblioteca argon2-cffi. A senha em si nunca é gravada.

Explicação. Argon2id é lento e usa bastante memória de propósito, o que torna caro testar muitas senhas mesmo com GPU. A conferência roda até para usuário inexistente, para não revelar pelo tempo quais usuários existem. O bug B4 mostrou que um hash corrompido podia levantar uma exceção inesperada; agora ela é tratada como "senha não confere".

Onde ver. `src/pulseops/security/passwords.py`; `src/pulseops/services/auth.py` linhas 33–38; testes de `tests/unit/test_passwords.py`.

Aprofundamento provável. "E se os parâmetros ficarem fracos com o tempo?" Pista: o argon2-cffi oferece `check_needs_rehash`; o PulseOps **não** usa. Seria uma melhoria.

Limite honesto. Não há limite de tentativas de login.

7.25 H25. Como funciona a proteção contra CSRF?

Resposta curta. Toda ação que altera dados exige um token secreto da sessão, no formulário ou no header `X-CSRF-Token`. Um site de terceiros consegue fazer o navegador enviar o cookie, mas não consegue ler o token.

Explicação. Somado a isso, o cookie de sessão é `SameSite=Lax`. O login também tem token próprio, num cookie separado, para impedir um "login forçado" na conta do atacante. A comparação é em tempo constante (`hmac.compare_digest`).

Onde ver. `src/pulseops/web/deps.py` linhas 80–95; testes `test_api_mutation_requires_csrf_header`, `test_form_action_requires_csrf_field`, `test_login_form_requires_its_csrf_cookie`.

Aprofundamento provável. "Se já há SameSite, por que o token?" Pista: defesa em profundidade; navegadores antigos e alguns cenários (subdomínios) não respeitam `SameSite` igual.

Limite honesto. Não foi feito um teste com navegador e um site malicioso de verdade; os testes enviam as requisições diretamente.

7.26 H26. Como o p95 é calculado?

Resposta curta. Pelo método *nearest-rank*: ordena as latências e pega a da posição `ceil(0,95 × n)`. No banco, `percentile_disc(0.95)`, que dá o mesmo valor.

Explicação. Existem vários métodos de percentil (alguns interpolam entre dois valores). O *nearest-rank* sempre devolve uma latência que realmente aconteceu. Um teste compara o SQL com a função Python para tamanhos de 1 a 137 amostras.

Onde ver. `src/pulseops/domain/metrics.py` linhas 8–19; `docs/decisoes/0011-metricas-e-percentil.md`; teste `test_sql_percentile_matches_nearest_rank`.

Aprofundamento provável. "Com 10 amostras, o p95 é o quê?" Pista: `ceil(9,5) = 10`, o maior valor. Com poucas amostras, o p95 é o máximo.

Limite honesto. Nada além do método documentado.

7.27 H27. Por que paginação por cursor?

Resposta curta. Porque o histórico cresce enquanto a pessoa navega. Com `OFFSET`, itens novos empurram os antigos e há repetição; com o cursor `(started_at, id)`, a próxima página começa exatamente depois do último item visto.

Explicação. Ver a parte F. O `id` entra como desempate porque dois checks podem ter o mesmo `started_at`.

Onde ver. `src/pulseops/services/queries.py` (`encode_cursor`, `list_checks`); teste `test_cursor_pagination_is_stable_and_complete`.

Aprofundamento provável. "Dá para pular para a página 50?" Pista: não; cursor só anda para a frente. É uma troca consciente.

Limite honesto. Nada a ressalvar.

7.28 H28. O que acontece se a configuração mudar durante um check?

Resposta curta. O resultado fica no histórico, mas não decide nada. Cada check guarda o `config_version` da reserva; se o monitor tiver outra versão quando o resultado chega, o efeito é descartado.

Explicação. Editar, pausar, retomar e arquivar incrementam a versão e zeram os contadores. O bug B6 era a pausa e a retomada não incrementarem a versão.

Onde ver. `src/pulseops/services/processing.py` linhas 164–167; `src/pulseops/services/monitors.py` linhas 171–177; testes `test_result_of_run_started_before_config_change_does_not_count` e `test_config_change_resets_counters_and_keeps_incident`.

Aprofundamento provável. "E o incidente aberto?" Pista: continua aberto e ganha um evento `config_changed` na linha do tempo.

Limite honesto. O nome `config_version` hoje significa "versão da contagem"; está documentado em RF03.5.

7.29 H29. Qual foi o bug mais interessante?

Resposta curta. O B8: com mais checks vencidos que vagas, o worker dormia o tick inteiro mesmo com vagas abrindo, e a vazão ficava em cerca de 20 checks por segundo. Nenhum teste funcional pegou; o ensaio de carga mostrou atraso mediano de 7,8 s.

Explicação. A correção: depois de preencher todas as vagas pedidas, o laço espera um **evento** de "vaga liberada", com o tick como limite. A primeira tentativa de correção estava errada (usava o número de tarefas em andamento); a condição certa é "reservei menos do que pedi". Depois da correção, a mediana caiu para 0,64 s.

Onde ver. `docs/registro-de-bugs.md` (B8); `src/pulseops/worker/runner.py` linhas 77–90; `docs/evidencias/ensaio-de-carga-antes-B8.md` e `docs/evidencias/ensaio-de-carga.md`.

Aprofundamento provável. “Como você teria achado sem o ensaio?” Pista: talvez não acharia; essa é a lição.

Limite honesto. O ensaio é local. Não diga “otimizei a performance em produção”.

7.30 H30. Como as migrações são validadas?

Resposta curta. Um teste aplica a migração num banco vazio e compara o resultado com os modelos (`compare_metadata`); outro faz `downgrade` até o fim e `upgrade` de novo. O CI repete o ciclo.

Explicação. Ver a parte F. O bug B1 (coluna no modelo e não na migração) foi pego por esse teste.

Onde ver. `tests/integration/test_migrations.py`; `.github/workflows/ci.yml`.

Aprofundamento provável. “Como renomear uma coluna sem parar o sistema?” Pista: em etapas (criar a nova, copiar, passar a usar, remover a antiga). O projeto não passou por isso.

Limite honesto. Há uma migração só. Evolução de esquema com dados não foi exercitada.

7.31 H31. O que é o padrão outbox?

Resposta curta. Gravar a intenção de enviar uma mensagem **na mesma transação** da mudança que a causou, e enviar depois por um processo separado. Assim a mensagem nunca se perde e nunca é enviada para uma mudança que foi desfeita.

Explicação. Ver a parte G. No PulseOps, `enqueue_incident_alert` é chamada dentro da transação de `record_result` e não faz commit sozinha (o comentário diz “quem chama decide”).

Onde ver. `src/pulseops/services/alerts.py` linhas 26–89; `docs/decisoes/0007-outbox-e-entrega-pelo-menos-uma-vez.md`; teste `test_outbox_event_is_atomic_with_incident`.

Aprofundamento provável. “E sem destino cadastrado?” Pista: o evento é gravado mesmo assim, sem entregas (`test_without_destination_event_is_still_recorded`).

Limite honesto. Nada a ressalvar além da duplicidade (H12).

7.32 H32. Como os logs evitam vazar segredos?

Resposta curta. Os logs são JSON, e antes de escrever cada campo o formatador mascara chaves sensíveis (senha, token, cookie…) e tira de URLs a *query string* e as credenciais.

Explicação. Uma URL monitorada pode ter `?api_key=...`. O payload do webhook também leva a URL mascarada. As explicações gravadas nos checks nunca contêm a URL (`test_explanation_never_contains_url_or_query`).

Onde ver. `src/pulseops/logs.py` linhas 14–32; testes de `tests/unit/test_logging.py`.

Aprofundamento provável. “E o caminho da URL?” Pista: o caminho fica; se houver segredo no caminho (`/token/abc`), ele aparece. É uma limitação.

Limite honesto. A máscara funciona por padrões; um segredo num formato inesperado pode passar.

7.33 H33. Como o CI está configurado?

Resposta curta. GitHub Actions com dois jobs: `qualidade` (ruff, mypy, migrações para cima e para baixo, pytest com cobertura mínima, conferência de referências dos documentos e auditoria de dependências) e `docker` (constrói a imagem e roda a demonstração inteira). O token do workflow só tem permissão de leitura.

Explicação. `permissions: contents: read` segue o princípio do menor privilégio: se uma dependência maliciosa rodasse no CI, não conseguiria escrever no repositório. O job `docker` prova que a instalação pelo Compose funciona do zero a cada push.

Onde ver. `.github/workflows/ci.yml`.

Aprofundamento provável. "As actions estão fixadas por hash?" Pista: não, por versão (`@v4`, `@v6`). Fixar por hash do commit seria mais seguro.

Limite honesto. Só afirme que o CI passa com o link de uma execução verde no GitHub. Sem isso, diga que ele foi validado localmente com `actionlint`.

7.34 H34. Qual foi o papel da IA no projeto?

Resposta curta. O código, os testes e a documentação foram produzidos com o Claude Code a partir de uma especificação que eu escrevi, com requisitos, regras de honestidade e critérios de aceitação. Eu estudei o código com este material e consigo explicar as decisões.

Explicação. Ser direta sobre isso evita constrangimento depois. O que a entrevistadora quer saber é se você entende o que está no repositório: mostre isso explicando uma decisão (outbox, SSRF, transações curtas) e respondendo os aprofundamentos.

Onde ver. README, seção "Como este projeto foi feito".

Aprofundamento provável. "Então o que você fez?" Pista: a especificação, a revisão, o estudo, a decisão de publicar, e o que você conseguir mostrar na hora, como alterar uma regra e atualizar os testes (exercícios da parte J).

Limite honesto. Não diga que escreveu o código sozinha.

8 Parte I — Como apresentar o projeto

8.1 Explicação de 30 segundos

> O PulseOps é um monitor de APIs HTTP que eu especifiquei e estudei a fundo. Ele verifica URLs periodicamente, separa resposta lenta de falha, só abre incidente depois de falhas seguidas e explica cada decisão. Os alertas saem por webhook com retentativas, sem atrasar as verificações. É Python com FastAPI, PostgreSQL, um worker separado, 251 testes com banco real e um laboratório que provoca as falhas e confere o comportamento em 11 passos.

Treine até sair natural, em voz alta. Se perguntarem “você que fez?”, responda como em H34.

8.2 Explicação de 2 minutos

1. **Problema (20 s).** Uma equipe fictícia descobria quedas pelos clientes e recebia alertas repetidos ou falsos. O PulseOps responde três perguntas: está no ar? está lento? por que o sistema decidiu isso?
2. **Como funciona (40 s).** São dois processos. A API serve o painel e a API JSON. O worker reserva no banco os monitores vencidos, faz as requisições fora de qualquer transação e grava o resultado aplicando a regra: 3 falhas seguidas abrem um incidente, 2 sucessos fecham. Cada mudança grava o motivo em texto.
3. **O que é difícil e como foi resolvido (40 s).**
   - Duplicidade: o banco proíbe dois incidentes abertos e dois checks em andamento por monitor, com índices únicos parciais.
   - Alertas: outbox na mesma transação, entrega pelo menos uma vez, com `event_id` estável para deduplicar.
   - Segurança: a política SSRF valida o IP na hora da conexão e conecta nesse IP, o que fecha o DNS rebinding.
4. **Como provar (20 s).** Testes com PostgreSQL real, CI, um ensaio de carga que achou um bug de vazão (B8) e o laboratório `scripts/demo.sh`.

8.3 Roteiro de demonstração de 5 minutos

Prepare antes: ambiente no ar com a demonstração (`scripts/demo.sh` já rodado uma vez), navegador aberto no painel e um terminal na raiz do projeto.

Tempo	O que fazer	O que dizer
0:00–0:30	Mostrar o README: diagrama e a seção do laboratório	“Dois processos e um banco. Vou mostrar as falhas acontecendo.”

Tempo	O que fazer	O que dizer
0:30–2:30	Rodar `scripts/demo.sh` no terminal	Comentar os passos conforme aparecem: verificações saudáveis; timeout; “falha 1 de 3, 2 de 3”; incidente aberto na 3ª; falhas seguintes não abrem outro; recuperação com 2 sucessos
2:30–3:30	No painel, abrir o incidente criado	Mostrar início versus confirmação, a linha do tempo com os motivos e o alerta com as tentativas
3:30–4:15	Página de notificações	“O receptor ficou fora do ar: aqui estão as tentativas com 503 e a espera crescente; quando voltou, entregou. As verificações não pararam.”
4:15–5:00	Abrir `src/pulseops/db/models.py` no índice `uq_incidents_one_open_per_monitor`	“Essa linha é o motivo de não haver incidente duplicado, mesmo com bug no código.”

Se algo falhar ao vivo, não esconda: o script diz `FALHOU: <motivo>` e termina com código 1. Explicar a falha é uma demonstração melhor do que fingir que não aconteceu. Tenha as capturas de `docs/imagens/` como plano B.

8.4 Descrição para currículo

Use só o que foi entregue e pode ser mostrado:

> **PulseOps** — projeto pessoal de estudo (Python, FastAPI, PostgreSQL, SQLAlchemy, Docker). Monitor de APIs HTTP com worker separado, regra de incidentes por limiares, alertas por webhook com outbox e retentativas e proteção contra SSRF. 251 testes automatizados com PostgreSQL real, CI no GitHub Actions e laboratório de falhas reproduzível. Desenvolvido com assistência de IA a partir de especificação própria. [link do repositório]

Variações curtas para o LinkedIn ou um formulário: “Monitor de APIs em Python com FastAPI e PostgreSQL, testes com banco real e laboratório de falhas”.

Não escreva: “em produção”, “usado por N pessoas”, “alta disponibilidade”, “escalável”, “experiência profissional”, ou uma porcentagem de melhoria de desempenho sem citar que foi num ensaio local.

8.5 O que mostrar primeiro a um recrutador

Recrutadores costumam ter poucos minutos. Na ordem:

1. **O README**, até a imagem do painel: o que é, em uma frase, e que funciona.
2. **A seção do laboratório de falhas** do README: mostra que o comportamento foi verificado, não só descrito.

3. **O selo ou a aba Actions** com o CI verde (só se o repositório estiver publicado e o CI tiver passado).
4. **O registro de bugs** (`docs/registro-de-bugs.md`): mostra processo, e é raro em projetos de portfólio.

Para uma pessoa técnica, acrescente: `docs/decisoes/` (as ADRs) e um teste que você saiba explicar de cor, como `test_rebinding_is_checked_at_connect_time`.

8.6 Afirmações que exigem evidência ou estudo antes

Afirmação	Antes de dizer, tenha
"O CI está verde"	o link de uma execução verde no GitHub
"Funciona com uma instalação do zero"	`docs/evidencias/instalacao-limpa.md` e, de preferência, ter feito você mesma
"Cobertura de 95%"	rodado `uv run pytest --cov` e visto o número, e saber o que cobertura não prova (parte G)
"Aguenta 200 monitores"	a frase completa: "num ensaio local, 200 monitores a cada 10 s, numa máquina de 4 CPUs"
"É seguro contra SSRF"	saber explicar as duas camadas, o DNS rebinding e o que **não** é coberto (firewall de saída)
"Não perde alertas"	saber explicar o outbox e por que pode **duplicar**
"Usei async"	saber explicar o event loop e por que um código bloqueante trava tudo (H17)
"Usei orientação a objetos"	exemplos concretos (H2), sem exagerar
Qualquer número	o arquivo de onde ele veio

Se não souber responder um aprofundamento, diga "não sei, mas é assim que eu descobriria", e aponte o arquivo ou o teste. É melhor que inventar.

9 Parte J — Aprendizagem ativa

Ler não basta para fixar. Os exercícios vão do mais simples ao mais difícil. Tente cada um **antes** de olhar o gabarito, que fica em uma seção separada logo depois desta parte. Use um ramo do Git para os exercícios que mexem no código (`git switch -c estudo`), e volte com `git switch -` e `git restore .` quando quiser descartar.

Para rodar trechos de Python no ambiente do projeto:

```
uv run python
```

Isso abre o interpretador interativo (`>>>`) com o PulseOps importável. Saia com `exit()`.

9.1 Nível 1 — Ler e localizar

J1. Em que arquivo está a lista de resultados possíveis de um check (`healthy`, `slow`...)? Quais deles contam como falha do alvo?

J2. Qual tabela guarda cada tentativa de envio de webhook? E qual guarda o motivo de cada mudança de saúde?

J3. Abra `compose.yaml`. Qual serviço roda as migrações, e por que a `api` só sobe depois dele?

J4. Encontre o teste que prova que o nome de um monitor é escapado no HTML. Que ataque ele previne?

9.2 Nível 2 — Prever o resultado antes de executar

Escreva a sua previsão num papel. Depois execute no `uv run python` e compare.

J5.

```
from pulseops.services.alerts import backoff_delay

print(backoff_delay(3, 5, 300), backoff_delay(10, 5, 300))
```

J6.

```
from pulseops.services.alerts import is_permanent_status

print(is_permanent_status(404), is_permanent_status(429), is_permanent_status(503))
```

J7.

```
from pulseops.domain.metrics import nearest_rank, ratio

print(nearest_rank([120, 80, 300, 95, 110], 95), nearest_rank([], 95), ratio(0, 0), ratio(3, 4))
```

J8. Para cada URL, a política aceita ou recusa? Se recusa, com qual motivo?

```
from pulseops.security.url_policy import DestinationPolicy

p = DestinationPolicy()
for url in [
    "http://0x7f.1/",
    "https://api.example.com:8443/x",
    "http://[::ffff:127.0.0.1]/",
    "http://example.com:5432/",
]:
    try:
        print(url, "aceita", p.validate_url(url))
    except ValueError as e:
        print(url, "recusada:", e)
```

J9. Com `failure_threshold=2` e `recovery_threshold=2`, a sequência é timeout, erro interno, timeout. Abre incidente? Em qual check?

J10. Com os mesmos limiares, a sequência é timeout, lento, timeout. Abre incidente?

J11.

```
from pulseops.domain.status_codes import parse_status_codes

codes = parse_status_codes("200-299,304")
print([c for c in (200, 204, 301, 304, 404) if c in codes])
```

9.3 Nível 3 — Alterar uma regra e atualizar os testes

J12. Hoje, uma resposta **lenta** conta como sucesso para encerrar um incidente (ADR 0004). Suponha que a equipe decidiu o contrário: durante um incidente, só respostas `healthy` contam para a recuperação; respostas lentas não somam nem zeram a contagem de sucessos.

1. Antes de mexer, rode `uv run pytest tests/unit/test_health.py` e anote quantos passam.
2. Escreva primeiro o teste novo (ele deve falhar).
3. Altere `src/pulseops/domain/health.py`.
4. Qual teste existente vai falhar, e por quê? Ele deve ser apagado, alterado ou mantido? Justifique pela regra R4 (não mudar resultado esperado só para passar).
5. Que documentos precisam mudar junto?

J13. Mude o padrão de `webhook_max_attempts` de 6 para 8. Quais testes, documentos e variáveis de ambiente você precisa conferir? (Dica: procure com `grep -rn "max_attempts\|MAX_ATTEMPTS" src tests docs compose*.yaml .env.example`.)

9.4 Nível 4 — Implementar uma pequena melhoria

J14. O SQLAlchemy 2.1 avisa que `Result.tuples()` está obsoleto (o aviso aparece ao rodar os testes). Encontre os usos em `src/` e troque pela forma recomendada. Rode a suíte inteira e confirme que o aviso sumiu e nada quebrou.

J15. Ao fazer login, o PulseOps não confere se o hash da senha foi gerado com parâmetros antigos. Use `PasswordHasher.check_needs_rehash` do argon2-cffi para, depois de um login bem-sucedido, regravar o hash quando necessário. Escreva um teste que cria um hash com parâmetros mais fracos, faz login e confere que o hash gravado mudou.

9.5 Nível 5 — Diagnosticar sem olhar a resposta

Para cada sintoma, escreva: hipótese mais provável, o comando que confirma e a correção.

J16. Depois de trocar `POSTGRES_PASSWORD` no `.env`, `docker compose up` termina com `migrate` em `exited (1)`.

J17. O painel mostra todos os monitores como "desatualizados", mas a página abre normalmente.

J18. Um monitor para `http://intranet.empresa.local/health` é recusado com "aponta para endereço privado".

J19. Na página de notificações, todas as entregas para um destino aparecem como `failed` depois de uma única tentativa, com status 404.

J20. `uv run pytest` falha logo no começo com `connection refused` na porta 5433.

J21. Um monitor está "lento" o tempo todo, mas a API responde em 300 ms quando você testa do seu computador.

9.6 Checklist de compreensão

Marque só o que você consegue **explicar em voz alta, sem ler**:

- [ ] O caminho de um check, da reserva à gravação, e por que são duas transações.
- [ ] O que é `run_key` e que problema resolve.
- [ ] A diferença entre `slow` e falha, e por que lento não abre incidente.
- [ ] O exemplo obrigatório (falha, falha, sucesso, falha × 3, sucesso × 2), com as datas de início e confirmação.
- [ ] Por que existem índices únicos **parciais**, e quais são os três.
- [ ] O que `FOR UPDATE SKIP LOCKED` faz.
- [ ] O padrão outbox e por que a entrega pode duplicar.
- [ ] As duas camadas da proteção SSRF e o que é DNS rebinding.
- [ ] Como o token de sessão é guardado e o que o token CSRF impede.
- [ ] Por que as métricas mostram "N/A", e qual é o denominador.
- [ ] O que acontece quando o worker reinicia.
- [ ] Como o painel mostra que o worker parou.
- [ ] Por que os testes usam PostgreSQL real e um relógio falso.
- [ ] Dois bugs do registro, com causa e lição.
- [ ] Três limitações do projeto e o que você faria em cada uma.

10 Gabarito comentado da parte J

As respostas dos níveis 2 a 4 foram **executadas** no commit explicado (a46ee88): as saídas abaixo são as que o código produziu, e as alterações dos exercícios J12, J14 e J15 foram feitas numa cópia descartável do repositório, com a suíte inteira rodando depois.

10.1 Nível 1

J1. `src/pulseops/domain/outcomes.py`. Contam como falha do alvo: `unexpected_status`, `timeout`, `connection_error`, `tls_error` e `blocked`. `healthy` e `slow` são sucesso; `internal_error` é falha do próprio PulseOps e não conta para nada.

J2. `delivery_attempts` (uma linha por tentativa, com status, duração e erro) e `health_changes` (de qual saúde para qual, quando e o `reason`).

J3. O serviço `migrate`. A `api` e o `worker` têm `depends_on: migrate: {condition: service_completed_successfully}`: só sobem depois de o `migrate` terminar com código 0. Assim, nunca rodam contra um banco sem as tabelas.

J4. `test_monitor_name_is_escaped` em `tests/integration/test_dashboard.py`. Previne **XSS** (*cross-site scripting*): um nome como `<script>...</script>` apareceria como texto, e não seria executado pelo navegador. Quem garante é o `autoescape=True` do Jinja (bug B3).

10.2 Nível 2

J5. `20.0 300.0`. A 3ª falha espera $5 \times 2^2 = 20$ s. A 10ª daria $5 \times 2^9 = 2560$ s, mas o teto é 300.

J6. `True False False`. 404 é erro do pedido (permanente). 429 (“muitas requisições”) é exceção explícita: vale tentar mais tarde. 503 não é 4xx, então é temporário.

J7. `300 None None 75.0`. Com 5 valores, a posição é $\lceil 0{,}95 \times 5 \rceil = 5$: o maior, 300. Lista vazia e denominador zero dão `None` (o “N/A” do painel), nunca 0.

J8.

```
http://0x7f.1/ recusada: endereço IP em forma não canônica
https://api.example.com:8443/x aceita Destination(scheme='https', host='api.example.com', port=8443)
http://[::ffff:127.0.0.1]/ recusada: ::ffff:127.0.0.1 aponta para endereço de loopback (::ffff:7f00:1)
http://example.com:5432/ recusada: porta 5432 não permitida (permitidas: 80, 443, 8080, 8443)
```

O terceiro caso é o mais traiçoeiro: um endereço IPv6 que “embrulha” um IPv4. A política extrai o IPv4 de dentro (`ipv4_mapped`) e confere de novo. A `UnsafeDestinationError` é uma subclasse de `ValueError`, por isso o `except ValueError` do exercício a captura.

J9. Abre, no **3º** check. O erro interno não soma nem zera:

```
timeout         none  degraded  falhas=1  "Falha 1 de 2 necessárias para abrir incidente"
internal_error  none  degraded  falhas=1  "Erro interno do PulseOps ignorado na contagem"
timeout         open  down      falhas=2  "Incidente confirmado após 2 falhas consecutivas"
```

J10. Não abre. Lento é **sucesso** para a contagem: zera as falhas.

```
timeout  none  degraded  falhas=1 sucessos=0
slow     none  degraded  falhas=0 sucessos=1  "Resposta lenta"
timeout  none  degraded  falhas=1 sucessos=0
```

Se a pergunta numa entrevista for “um serviço que alterna entre lento e timeout nunca abre incidente?”, a resposta honesta é “com esses limiares, não; ele fica degradado o tempo todo, e isso aparece no painel”. É uma consequência da ADR 0004.

J11. `[200, 204, 304]`. A faixa `200-299` cobre 200 e 204; 304 está listado à parte; 301 e 404 não.

10.3 Nível 3

J12. Resultado obtido na cópia:

1. Antes: `17 passed`.
2. O teste novo, escrito primeiro, falhou como esperado:

```python
def test_slow_does_not_count_towards_recovery_during_incident() -> None:
    state, decisions = run([F, S, Outcome.SLOW, S], failure=1, recovery=2)
    assert actions(decisions)[2] is IncidentAction.NONE  # lento não soma
    assert decisions[2].state.consecutive_successes == 1  # nem zera
    assert actions(decisions)[3] is IncidentAction.RESOLVE
```

3. A alteração, no começo do ramo `if state.incident_open:` de `_on_success`:

```python
if obs.outcome is Outcome.SLOW:
    reason = f"Resposta lenta não conta para encerrar o incidente: {obs.explanation}."
    return Decision(replace(state, health=Health.DOWN), IncidentAction.NONE, state.health, reason)
```

`replace(state, ...)` parte do estado **anterior**, então os contadores não mudam.

4. Na suíte inteira: `1 failed, 251 passed`. Falhou `test_slow_response_counts_towards_recovery`, que afirma a regra antiga. Aqui mudar o teste é correto, porque a **regra mudou de propósito**: ele deve ser substituído pelo teste novo, e o motivo vai na mensagem do commit. A regra R4 proíbe outra coisa: mudar o esperado de um teste para esconder um bug, sem que a regra tenha mudado. Repare também que o trecho que deixava o monitor `degraded` ao se recuperar com resposta lenta virou código morto e pode sair.
5. Documentos: `docs/decisoes/0004-lentidao-nao-abre-incidente.md` (registrar a nova decisão, ou uma ADR nova que substitui a 0004), RF03 em `docs/requisitos.md`, a tabela de regras do README e este guia (partes A e H9).

J13. O teste `test_attempts_are_limited_then_failed` usa `settings.webhook_max_attempts`, então continua certo sozinho. Mudam: `src/pulseops/config.py` (padrão), `.env.example`, a tabela de variáveis em `docs/operacao.md`, RF06.5 em `docs/requisitos.md` e o texto da parte G e de H11 deste guia. O `compose.demo.yaml` define 20 para a demonstração e não muda. Rode `uv run python scripts/conferir_referencias.py` e a suíte.

10.4 Nível 4

J14. Os dois usos estão em `src/pulseops/services/queries.py` (linhas 119 e 329). A forma recomendada é iterar o resultado direto, porque cada linha já se comporta como tupla:

```
counts = {status: count for status, count in rows}
```

Na cópia: `251 passed`, mypy sem erros, e o número de avisos caiu de 74 para 0. Não use `dict(rows)`: esse é exatamente o bug B2.

J15. Na cópia, a solução abaixo passou na suíte e no mypy, e o teste novo **falhou antes** da alteração (o hash gravado continuava `m=8192,t=1,p=1`):

```
# security/passwords.py
def needs_rehash(password_hash: str) -> bool:
    """O hash foi feito com parâmetros diferentes dos atuais (por exemplo, mais fracos)."""
    return _hasher.check_needs_rehash(password_hash)


# services/auth.py, dentro de authenticate
if verify_password(operator.password_hash if operator else None, password):
    if operator is not None and needs_rehash(operator.password_hash):
        operator.password_hash = hash_password(password)
    return operator
```

```
async def test_weak_hash_is_upgraded_on_login(sessions, operator, app_client) -> None:
    weak = PasswordHasher(time_cost=1, memory_cost=8192, parallelism=1).hash(ADMIN_PASSWORD)
    async with sessions.begin() as s:
        await s.execute(update(Operator).values(password_hash=weak))
    await login(app_client, ADMIN_USER, ADMIN_PASSWORD)
    async with sessions() as s:
        stored = await s.scalar(select(Operator.password_hash))
    assert stored != weak
    assert "m=65536" in stored
```

Funciona porque a rota de login já faz `commit()` depois de criar a sessão, e a alteração do hash vai junto na mesma transação. Ponto de atenção: `hash_password` recusa senhas com menos de 12 caracteres; como o cadastro já exige 12, isso não acontece com operadoras criadas pelo PulseOps.

10.5 Nível 5

J16. O volume `pgdata` foi criado com a senha **antiga**; o PostgreSQL só lê `POSTGRES_PASSWORD` na primeira inicialização. Confirme com `docker compose logs migrate` (erro de autenticação). Correção: voltar à senha antiga, ou trocar a senha dentro do banco (`ALTER USER`), ou apagar o volume com `docker compose down -v`, **perdendo os dados**.

J17. O worker está parado ou sem acesso ao banco. Confirme com `docker compose ps worker` e `docker compose logs worker`; o painel também mostra o aviso “Worker sem sinal”. Correção: `docker compose start worker` (ou resolver o erro que aparece no log).

J18. O nome resolve para um IP privado (por exemplo, `10.x.x.x`), e a política SSRF recusa de propósito. Não é um bug. O PulseOps foi feito para monitorar endereços públicos; a única exceção é a lista do modo demonstração, que não deve ser usada para contornar a política fora do laboratório.

J19. 404 é tratado como **erro permanente** (`is_permanent_status`), então não há retentativa. A causa mais provável é a URL do destino errada (caminho trocado). Confira a URL na página de notificações e teste com `curl -X POST <url> -d '{}'`. Depois de corrigir o destino, os **próximos** eventos serão entregues; os que já ficaram `failed` não são reenviados.

J20. O banco de testes não está rodando. `docker ps -a` mostra `pulseops-testdb` parado ou inexistente. Correção: `docker start pulseops-testdb`, ou o `docker run` da parte B.

J21. O tempo medido é o do **worker**, não o do seu computador, e inclui DNS, conexão e TLS. Se o worker estiver em outra rede, ou se a conexão não é reaproveitada (o PulseOps abre uma conexão nova a cada check, com `max_keepalive_connections=0`), o tempo pode passar dos 300 ms que você vê no navegador. Confira a duração dos checks no histórico e compare com o limite do monitor; ajuste o limite se ele estiver abaixo do custo real de uma conexão nova.